

Spring Boot essentials exam-oriented explanation with important exam questions

Below is **Spring Boot Essentials — Exam-oriented, concise, yet detailed notes** with important exam questions.

Perfect for college exams, viva, and interviews.

SPRING BOOT ESSENTIALS — EXAM-ORIENTED NOTES

1. Introduction to Spring Boot

What is Spring Boot?

Spring Boot is a framework built on top of Spring that simplifies application development by:

- Eliminating boilerplate configuration
- Providing auto-configuration
- Offering embedded servers (Tomcat/Jetty/Undertow)
- Reducing XML configuration (almost zero XML)

Why Spring Boot? (Advantages)

- Auto-configuration
- Starter dependencies
- Embedded server
- Production-ready features (metrics, health checks)
- Easy to create standalone executable JARs
- Rapid application development (RAD)

Core Features

1. Auto Configuration
2. Spring Boot Starter Dependencies
3. Spring Boot CLI
4. Actuator
5. Embedded Servers

2. Spring Boot Architecture

Main Components

1. Starter Dependencies

- Bundles required libraries
- Examples:
 - `spring-boot-starter-web`
 - `spring-boot-starter-data-jpa`
 - `spring-boot-starter-thymeleaf`

2. Auto Configuration

- Configures beans automatically based on classpath.
- E.g., If Spring MVC exists → `DispatcherServlet` auto-configured.

3. Spring Boot Application Class

java

 Copy code

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

4. Embedded Server

Spring Boot apps run directly via

```
java -jar app.jar
```

3. @SpringBootApplication Annotation

It is a combination of:

- `@Configuration` – defines beans
- `@EnableAutoConfiguration` – enables auto-config
- `@ComponentScan` – scans components

4. Spring Boot Starters

Common Starters

Starter	Use
spring-boot-starter-web	To build REST APIs and MVC
spring-boot-starter-data-jpa	ORM, Hibernate + JPA
spring-boot-starter-test	Testing
spring-boot-starter-security	Authentication/Authorization
spring-boot-starter-thymeleaf	UI template engine

5. Application Properties

Used for configuration.

Types

- `application.properties`
- `application.yml`

Examples

ini

 Copy code

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost/test
logging.level.org.springframework=DEBUG
```

6. Embedded Servers

Spring Boot supports:

- Tomcat (default)
- Jetty
- Undertow

Advantage:

- No need for manual deployment on external servers (WAR file).

7. Building REST API in Spring Boot

REST Controller

java

 Copy code

```
@RestController
@RequestMapping("/api")
public class StudentController {

    @GetMapping("/students")
    public List<Student> getAll() {
        return service.getAllStudents();
    }
}
```

8. Dependency Injection in Spring Boot

Ways

- Constructor Injection (recommended)
- Setter Injection
- Field Injection (not preferred)

9. Spring Boot with JPA (ORM)

Entity Example

java

 Copy code

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
}
```

Repository

java

@Repository

```
public interface UserRepo extends JpaRepository<User, Long> {}
```

10. Spring Boot Actuator

Provides production monitoring tools:

- /actuator/health
- /actuator/info
- /actuator/metrics

Need to add:

 Copy code

```
spring-boot-starter-actuator
```

11. Spring Boot AOP (Aspect Oriented Programming)

Helps in:

- Logging
- Transaction management
- Security
- Profiling

Annotation:

- @Aspect
- @Before
- @After
- @Around

12. Spring Boot Security (Basics)

Default:

- Uses Basic Authentication
- Default user is **user**
- Password shown in console

To configure:

```
pgsql
```

[Copy code](#)

```
spring.security.user.name=admin  
spring.security.user.password=1234
```

13. Profiles in Spring Boot

Used for environment-specific configs:

- dev
- test
- prod

```
ini
```

[Copy code](#)

```
spring.profiles.active=dev
```

14. Spring Boot DevTools

Enables:

- Auto restart
- Live reload
- Fast development

Dependency:

[Copy code](#)

```
spring-boot-devtools
```

★ IMPORTANT 15 MARK & VIVA QUESTIONS

1. Explain Spring Boot architecture with diagram.
2. What is auto-configuration? How does it work?
3. Difference between Spring and Spring Boot.

4. What are Spring Boot starter dependencies? Why are they useful?
 5. Explain `@SpringBootApplication` annotation.
 6. What is the purpose of `application.properties` / `yml`?
 7. Explain embedded servers in Spring Boot.
 8. How to create a REST API in Spring Boot?
 9. Explain Spring Data JPA components — Entity, Repository, Service.
 10. What is Spring Boot Actuator and its endpoints?
 11. Explain dependency injection in Spring Boot.
 12. Explain AOP (Aspect Oriented Programming) with Spring Boot examples.
 13. How do you secure a Spring Boot application?
 14. What are Spring Boot profiles and why are they used?
 15. Explain the lifecycle of a Spring Boot application.
-

★ SHORT ANSWER (3–5 Marks) Questions

- What is Spring Boot CLI?
 - What are the advantages of Spring Boot?
 - What is the difference between `application.properties` and `application.yml`?
 - What is Spring Starter Parent?
 - Explain the meaning of `@RestController` .
 - What is the purpose of `SpringApplication.run()` ?
 - What is the use of H2 database in Spring Boot?
-

If you want, I can prepare:

- ✓ Previous Year Question Bank
- ✓ Handwritten-style PDF Notes
- ✓ Important Diagrams (Architecture, Flow, Layers)

Just tell me!



Need of Springboot

- Spring Boot Overview
- Basic Introduction of MAVEN exam-oriented explanation with important exam questions

Below is **Need of Spring Boot, Spring Boot Overview, and Basic Introduction to Maven** — all explained in **exam-oriented style** with **important exam questions** (3–5 marks & 10–15 marks).

✓ 1. Need of Spring Boot (Why Spring Boot?)

Exam-Oriented Explanation

Traditional Spring framework requires:

- Complex XML configurations
- Manual dependency management
- Deployment on external servers
- Time-consuming setup for each module

Spring Boot was introduced to solve these problems and **make application development faster, easier, and production-ready**.

Why do we need Spring Boot? (Key Points)

1. Removes Boilerplate Configuration

- No XML configuration
- Auto-configures beans based on classpath

2. Embedded Servers

- Comes with Tomcat/Jetty built-in
- No need to deploy WAR on external servers

3. Starter Dependencies

- Automatically pulls required libraries

4. Production-Ready Features

- Health checks, metrics, monitoring via Actuator

5. Rapid Application Development (RAD)

- Create apps quickly with minimal code

6. Easy Integration

- Works easily with JPA, Security, Web, JMS, Kafka, etc.

7. Convention over Configuration

- Defaults provided to reduce developer decisions

★ Important Exam Questions

Long Answers (10–15 marks)

- Q1. Explain the need for Spring Boot. How does it simplify Java application development?
- Q2. Why was Spring Boot introduced? Discuss the problems in traditional Spring solved by Spring Boot.

Short Answers (3–5 marks)

- Q1. List any four advantages of Spring Boot.
- Q2. What is Auto Configuration?

✅ 2. Spring Boot Overview

Definition

Spring Boot is a framework built on top of Spring that provides:

- ✓ Auto-configuration
- ✓ Starter dependencies
- ✓ Embedded web servers
- ✓ Production-ready features
- ✓ Standalone, executable applications

Key Features (Exam-Oriented)

1. Auto Configuration

- Automatically configures beans based on dependencies present.

2. Starter Dependencies

- Makes dependency management easy.

Example:

- `spring-boot-starter-web`
- `spring-boot-starter-data-jpa`

3. Spring Boot CLI

- Command-line tool to run Groovy-based scripts.

4. Embedded Servers

- Tomcat is default.
- Supports Jetty, Undertow.

5. Actuator

- Provides endpoints like `/health`, `/metrics`, `/info`.

6. Production-Ready

- Logging, monitoring, custom health indicators.

7. Simplified Architecture

- Uses `@SpringBootApplication` annotation.

★ Important Exam Questions

Long Answers (10–15 marks)

- Q1. Explain Spring Boot architecture and key features.
- Q2. Describe the components of Spring Boot with neat diagram.

Short Answers (3–5 marks)

- What is Spring Boot?
- What are Spring Boot starters?
- What is `@SpringBootApplication` ?

✅ 3. Basic Introduction to MAVEN

Exam-Oriented Explanation

What is Maven?

Maven is a **build automation and project management tool** used primarily for Java projects.

It simplifies:

- Dependency management
- Project build (compilation, packaging)
- Standard project structure
- Running tests

Why Maven? (Need)

1. Automatic Dependency Management

- No manual JAR downloading

2. Standard Directory Structure

- `src/main/java`
- `src/test/java`

3. Easy Build Commands

- `mvn clean`
- `mvn install`

4. Plugins Support

- For building WAR/JAR
- Running Spring Boot apps
- Code quality checks

5. Integration with Repositories

- Central Repository
- Local Repository
- Remote Repository

Maven Project Structure

CSS

 Copy code

myproject

```
|— src
|   |— main
|       |— java
|       |— resources
|   |— test
|— pom.xml
```

What is `pom.xml` ?

POM = Project Object Model

Contains:

- Project info
- Dependencies
- Plugins
- Build settings

Example:

xml

 Copy code

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

★ Important Exam Questions

Long Answers (10–15 marks)

- Q1. What is Maven? Explain Maven architecture and POM file.
- Q2. Explain dependency management in Maven with an example.

Short Answers (3–5 marks)

- What is pom.xml ?
- What is Maven dependency management?
- What are Maven repositories?

If you want, I can prepare:

- ✓ Handwritten Style Notes (PDF)
- ✓ Spring Boot + Maven Combined 10 Marks Answers
- ✓ MCQs for exam practice
- ✓ Diagram for Spring Boot Architecture

Just tell me!

     ...

Building Spring Webapplication with Boot exam-oriented
explanation with important exam questions

Below is “Building Spring Web Application with Spring Boot” in a perfect exam-oriented explanation with important 3–5 mark and 10–15 mark questions.

This covers architecture, annotations, flow, layers, and example code — everything needed for theory exams, viva, and practicals.

✓ Building Spring Web Application with Spring Boot

Exam-Oriented Explanation

Spring Boot makes web application development extremely simple using **Spring MVC**, **embedded Tomcat**, and **auto-configuration**.

A Spring Boot web app typically exposes:

- ✓ Web pages (Thymeleaf/JSP)
- ✓ REST APIs
- ✓ Controller-Service-Repository layers

1. Required Steps to Build Spring Web Application in Spring Boot

Step 1: Create Spring Boot Project

Using **Spring Initializr**:

Choose dependencies:

- `spring-boot-starter-web`
- `spring-boot-starter-thymeleaf` (for UI)
- `spring-boot-starter-data-jpa` (database)
- `mysql-connector-j` (if using MySQL)

This generates:

- `src/main/java` → source code
- `src/main/resources` → templates/static
- `application.properties`

Step 2: Application Entry Point

Every Spring Boot app has a class with `@SpringBootApplication` .

java

 Copy code

```
@SpringBootApplication
public class WebAppApplication {
    public static void main(String[] args) {
        SpringApplication.run(WebAppApplication.class, args);
    }
}
```

Step 3: Create Controller (Spring MVC Component)

Controller handles client requests.

Example Controller

java

 Copy code

```
@Controller
public class HomeController {

    @GetMapping("/")
    public String homePage(Model model) {
        model.addAttribute("message", "Welcome to Spring Boot Web App");
        return "index"; // Returns index.html (Thymeleaf)
    }
}
```

Step 4: UI with Thymeleaf (View Layer)

src/main/resources/templates/index.html

html

 Copy code

```
<!DOCTYPE html>
<html>
<body>
<h2 th:text="${message}"></h2>
```

```
</body>
</html>
```

Spring Boot auto-configures Thymeleaf and resolves the view automatically.

Step 5: Service Layer (Business Logic)

java

 Copy code

```
@Service
public class StudentService {

    @Autowired
    private StudentRepository repo;

    public List<Student> getAllStudents() {
        return repo.findAll();
    }
}
```

Step 6: Repository Layer (Database Layer using JPA)

java

 Copy code

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Integer> {}
```

Step 7: Model / Entity Class

java

 Copy code

```
@Entity
public class Student {
    @Id
    @GeneratedValue
    private int id;
    private String name;
```

```
private String city;  
}
```

Step 8: application.properties Configuration

ini

 Copy code

```
server.port=8081  
  
spring.datasource.url=jdbc:mysql://localhost:3306/testdb  
spring.datasource.username=root  
spring.datasource.password=1234  
  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```

Step 9: Running the Application

Run:

arduino

 Copy code

```
mvn spring-boot:run
```

or

```
java -jar WebAppApplication.jar
```

Embedded Tomcat starts automatically → no need to install external server.

Spring Boot Web Application Architecture (Important for Exams)

less

 Copy code

```
Client (Browser)  
↓  
Controller (@Controller or @RestController)  
↓
```


Service Layer (@Service)
↓
Repository Layer (JPA @Repository)
↓
Database (MySQL/H2)

Important Spring Boot Web Annotations

Annotation	Use
@Controller	Handles web requests and returns views
@RestController	Returns JSON instead of views
@GetMapping , @PostMapping	Map URLs to methods
@Autowired	Dependency injection
@Entity	Represents database table
@Repository	DAO layer
@Service	Business logic layer

★ Exam Questions (Very Important)

10–15 Marks Questions (Long Answers)

Q1. Explain the steps to build a Spring Boot web application.

(Include Project structure + Controller + View + Service + Repository + application.properties)

Q2. Describe the architecture of Spring Boot Web Application with a neat diagram.

Q3. Explain the role of Controller, Service, and Repository in Spring Boot MVC.

Q4. Write a short note on Spring Boot Starter Web. What components does it provide?

Q5. Explain Spring MVC flow in a Spring Boot Web Application.

3–5 Marks Questions (Short Answers)

Q1. What is the use of `spring-boot-starter-web` ?

Q2. What is the role of `@SpringBootApplication` ?

Q3. Difference between `@Controller` and `@RestController` .

Q4. What is the purpose of `application.properties` ?

Q5. What is view resolver in Spring Boot?

(Spring Boot auto-configures `ThymeleafViewResolver`)

Q6. What is the role of embedded Tomcat in Spring Boot?

★ If you want, I can also provide:

- ✓ Full 15 Marks Answer (Handwritten-style)
- ✓ Diagram PDF for Spring Boot Web Architecture
- ✓ Spring Boot Web Application Viva Questions
- ✓ Mini Project Example (CRUD App)

Just tell me!



Spring Boot in detail (Use Spring Boot for all demo & assignments here onwards)

□ Running a web application using Spring Boot with CRUD (with Static Data not DB)

Below is **Spring Boot** in detail + **Running a Spring Boot Web Application with FULL CRUD using Static Data (NO Database)** — perfectly explained for **exams, assignments, viva & practicals**.

This is **exactly the format teachers expect**.

SPRING BOOT — DETAILED EXAM-ORIENTED NOTES

1. What is Spring Boot?

Spring Boot is a framework built on top of Spring that:

- Eliminates complex configuration
- Provides ready-made setup (auto-configuration)
- Uses starter dependencies
- Runs with embedded servers (Tomcat)
- Helps build web applications easily

Spring Boot = **Simplified + Fast + Production-ready Spring**

2. Features of Spring Boot (Exam Points)

1. Auto Configuration

Automatically configures Spring beans based on classpath.

Example:

If `spring-boot-starter-web` is added → `DispatcherServlet` auto-configured.

2. Starter Dependencies

Bundles of commonly used libraries.

Examples:

- `spring-boot-starter-web`
- `spring-boot-starter-data-jpa`

3. Embedded Server

Built-in Tomcat → run directly using

```
java -jar app.jar
```

4. Production Ready (Actuator)

Health, metrics, monitoring → `/actuator/*`

5. Zero XML

Only annotations.

6. Easy project structure

Entry class + Controller + Service + Data.

3. Spring Boot Architecture (IMPORTANT)

less

 Copy code

Client (Browser / API request)

↓

Controller (`@RestController`)

↓

Service Layer (`@Service`)

↓

Repository (Static Data or DB)

↓

Model (Entity Class)

4. Spring Boot Project Structure

CSS

 Copy code

`src/main/java`

└─ `com.example.demo`

└─ `DemoApplication.java`

└─ `controller`

└─ `service`

└─ `model`

```
src/main/resources
└─ application.properties
```



ASSIGNMENT + DEMO

Spring Boot CRUD Web Application (NO Database — STATIC DATA)

This is extremely important for **practicals, exams, viva and assignments**.

- ✓ CRUD = Create, Read, Update, Delete
- ✓ Using **in-memory List** (No database)
- ✓ Using Spring MVC + REST Controller



Step 1: Create Spring Boot Project

Dependencies required:

- spring-boot-starter-web



Step 2: Application Starter Class

java

Copy code

```
@SpringBootApplication
public class CrudDemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(CrudDemoApplication.class, args);
    }
}
```



Step 3: Create Model Class

java

Copy code

```
public class Student {
    private int id;
```

```
private String name;
private String city;

// constructors, getters, setters
}
```

Step 4: Create Service with STATIC DATA

java

 Copy code

```
@Service
public class StudentService {

    // Static in-memory List
    private List<Student> students = new ArrayList<>(
        Arrays.asList(
            new Student(1, "Yuvraj", "Jalna"),
            new Student(2, "Rohan", "Pune"),
            new Student(3, "Sneha", "Mumbai")
        )
    );

    public List<Student> getAll() {
        return students;
    }

    public Student getById(int id) {
        return students.stream()
            .filter(s -> s.getId() == id)
            .findFirst()
            .orElse(null);
    }

    public void add(Student s) {
        students.add(s);
    }

    public void update(int id, Student updated) {
        for (int i = 0; i < students.size(); i++) {
            Student s = students.get(i);
            if (s.getId() == id) {
```

```
        students.set(i, updated);
        return;
    }
}

public void delete(int id) {
    students.removeIf(s -> s.getId() == id);
}
}
```

Step 5: Create Controller

java

 Copy code

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @Autowired
    private StudentService service;

    // READ ALL
    @GetMapping
    public List<Student> getAll() {
        return service.getAll();
    }

    // READ One
    @GetMapping("/{id}")
    public Student getOne(@PathVariable int id) {
        return service.getById(id);
    }

    // CREATE
    @PostMapping
    public String add(@RequestBody Student s) {
        service.add(s);
        return "Student added!";
    }
}
```

```
// UPDATE
@PutMapping("/{id}")
public String update(@PathVariable int id, @RequestBody Student s) {
    service.update(id, s);
    return "Student updated!";
}

// DELETE
@DeleteMapping("/{id}")
public String delete(@PathVariable int id) {
    service.delete(id);
    return "Student deleted!";
}
}
```

Step 6: Test Using Postman or Browser

Operation	URL	Method
Get all students	/students	GET
Get one student	/students/1	GET
Add student	/students	POST
Update student	/students/1	PUT
Delete student	/students/1	DELETE

Most Important Exam Questions

10–15 Marks Questions

1. Explain how to build a Spring Boot CRUD application using static data.

(Write steps + architecture + controller + service + code)

2. Describe the role of Controller, Service & Model in Spring Boot.

3. Write a detailed note on Spring Boot features and architecture.
 4. Explain the complete flow of a Spring Boot Web Application with diagram.
 5. Explain RESTful web services in Spring Boot with CRUD example.
-

3–5 Marks Questions

1. What is Spring Boot starter web?
 2. What is the use of `@RestController` ?
 3. What is `@SpringBootApplication` ?
 4. What is dependency injection in Spring Boot?
 5. What is difference between `GET` and `POST` ?
 6. What is static data? Why used in assignment?
-

★ I can also provide:

- ✓ Full handwritten-style PDF notes
- ✓ A mini-project folder with full CRUD
- ✓ MCQs + Viva Questions

Just tell me and I will prepare them!



Spring Data Module

□ Spring Data JPA(Repository support for JPA)

Below is **Spring Data Module + Spring Data JPA** explained in **exam-oriented** style with **important 3–5 mark and 10–15 mark questions**, plus clear examples.

✓ **SPRING DATA MODULE — EXAM ORIENTED NOTES**

Spring Data is a framework in the Spring ecosystem designed to **simplify data access** (CRUD, queries, pagination) for different data stores.

Spring Data provides support for:

- Relational databases (JPA, JDBC)
- NoSQL databases (MongoDB, Redis, Cassandra)
- Search engines (Elasticsearch)
- Cloud data sources

Main Goal of Spring Data

- ➔ To reduce boilerplate code in DAO layer
- ➔ To provide ready-made repository interfaces
- ➔ To offer easy query creation using method names

★ Key Features of Spring Data

1. Repository Abstraction

You only define an interface → Spring creates implementation automatically.

2. CRUD operations without writing SQL/DAO code

Methods like:

- `save()`
- `findById()`
- `findAll()`
- `deleteById()`

3. Query Methods using Method Names

Example:

- `findByName(String name)`
- `findByCity(String city)`

4. Pagination & Sorting Support

- `Pageable`
- `Sort`

5. Integration with JPA & Hibernate

Automatically interacts with ORM.

✓ SPRING DATA JPA (Repository Support for JPA)

Exam-Oriented Detailed Explanation

Spring Data JPA is a sub-module of Spring Data that simplifies the use of JPA (Java Persistence API).

It provides:

- Built-in CRUD operations
- Custom finder methods
- No need to write implementation classes
- Automatic query generation
- Excellent integration with Spring Boot

★ 1. Spring Data JPA Architecture (VERY IMPORTANT)

SCSS

 Copy code

Controller



Service



Spring Data JPA Repository



JPA (Hibernate)



Database (MySQL / H2 / PostgreSQL)

★ 2. Key Components of Spring Data JPA

1. Entity Class

Represents a database table.

java

 Copy code

```
@Entity
public class Student {
    @Id
    @GeneratedValue
    private int id;
    private String name;
```

```
    private String city;  
}
```

2. Repository Interface

Spring Data JPA provides these main repository types:

✓ CrudRepository

Basic CRUD operations.

✓ JpaRepository (Most Used)

Extends CrudRepository + extra methods:

- findAll(Sort sort)
- findAll(Pageable pageable)

Example Repository

java

 Copy code

```
@Repository  
public interface StudentRepository extends JpaRepository<Student, Integer> {  
  
    // Custom query derived from method name  
    List<Student> findByCity(String city);  
  
    List<Student> findByNameContains(String name);  
}
```

Spring Boot automatically creates implementation for this interface.

3. Service Layer

java

 Copy code

```
@Service  
public class StudentService {  
  
    @Autowired  
    private StudentRepository repo;
```

```
public List<Student> getAll() {
    return repo.findAll();
}

public Student getById(int id) {
    return repo.findById(id).orElse(null);
}

public List<Student> getByCity(String city) {
    return repo.findByCity(city);
}
}
```

4. Controller Layer

java

 Copy code

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @Autowired
    private StudentService service;

    @GetMapping
    public List<Student> getAll() {
        return service.getAll();
    }

    @GetMapping("/{id}")
    public Student getOne(@PathVariable int id) {
        return service.getById(id);
    }
}
```

★ 3. application.properties (Important)

ini

 Copy code

```
spring.datasource.url=jdbc:mysql://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=1234

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

ddl-auto values:

- create → drops & creates table
- update → updates table
- validate → validates schema
- none → nothing

★ 4. Query Methods in Spring Data JPA

Spring generates queries based on method names.

Examples:

- `findByName(String name)`
- `findByCity(String city)`
- `findByAgeGreaterThan(int age)`
- `findByNameAndCity(String n, String c)`
- `findTop3ByOrderByAgeDesc()`

No SQL required!

★ 5. Custom Queries Using @Query

You can write JPQL or Native SQL.

JPQL Example:

java

 Copy code

```
@Query("select s from Student s where s.city = ?1")
List<Student> findStudentsByCity(String city);
```

Native SQL Example:

java

```
@Query(value="SELECT * FROM student WHERE city=?1", nativeQuery=true)  
List<Student> findByCityNative(String city);
```

★ 6. Pagination & Sorting

java

 Copy code

```
Pageable p = PageRequest.of(0, 5); // Page 0, Size 5  
Page<Student> page = repo.findAll(p);
```

Sorting:

java

 Copy code

```
List<Student> list = repo.findAll(Sort.by("name"));
```

★ 7. Advantages of Spring Data JPA

- Removes boilerplate DAO code
- Easy CRUD operations
- Method name-based queries
- Built-in pagination & sorting
- Integrates with Spring Boot seamlessly
- Less code, faster development



IMPORTANT EXAM QUESTIONS

★ 10–15 MARK QUESTIONS

1. Explain Spring Data JPA architecture with a neat diagram.
2. Describe Spring Data repository abstraction. What are CrudRepository and JpaRepository?

3. Explain query methods in Spring Data JPA with examples.
 4. What is the role of @Entity, @Repository, @Service annotations in Spring Data JPA?
 5. Explain how Spring Data JPA reduces boilerplate DAO code.
 6. Explain the working of Spring Boot + Spring Data JPA in a CRUD application.
-

★ 3–5 MARK QUESTIONS

- What is Spring Data?
 - Define Spring Data JPA.
 - What is JpaRepository?
 - What is a query method?
 - What is the purpose of @Query annotation?
 - What is the use of `spring.jpa.hibernate.ddl-auto` ?
 - Difference between CrudRepository and JpaRepository.
 - What is @Entity?
-

★ If you want, I can also prepare:

- ✓ Spring Data JPA full CRUD mini project
- ✓ Handwritten-style PDF notes
- ✓ MCQs and Viva Questions
- ✓ Diagram-only exam sheet

Just tell me!



Crud Repository & JPA Repository

Below is a clear, exam-oriented explanation of **CrudRepository** & **JpaRepository** with definitions, differences, diagrams, examples, and most important exam questions.

✓ CrudRepository & JpaRepository (Spring Data JPA)

Exam-Oriented Explanation

Spring Data JPA provides multiple repository interfaces to simplify database operations. The two most important are:

1 CrudRepository

Definition

CrudRepository is the **basic Spring Data interface** for performing **CRUD operations** on a database entity.

It provides methods like:

Method	Description
<code>save()</code>	Insert or update an entity
<code>findById()</code>	Retrieve a record by ID
<code>findAll()</code>	Retrieve all records
<code>deleteById()</code>	Delete a record by ID
<code>delete()</code>	Delete a record
<code>count()</code>	Count all rows

When to Use CrudRepository?

- ✓ When you only need **basic CRUD**
- ✓ When advanced operations (pagination, sorting, JPA features) are *not* required

Example

java

 Copy code

```
public interface StudentRepository extends CrudRepository<Student, Integer> {  
}
```

Using:

java

 Copy code

```
studentRepository.save(new Student(1, "Yuvraj", "CSE"));  
studentRepository.findById(1);  
studentRepository.findAll();  
studentRepository.deleteById(1);
```

2 JpaRepository

Definition

JpaRepository extends CrudRepository and PagingAndSortingRepository .
It provides **CRUD + Pagination + Sorting + JPA-Specific Features**.

Extra Features in JpaRepository

Feature	Description
findAll(Sort sort)	Sorting support
findAll(Pageable pageable)	Pagination support
flush()	Synchronize persistence context with DB
saveAndFlush()	Save and immediately flush
Batch operations	More optimized JPA batch operations

When to Use JpaRepository?

- ✓ Complex applications
- ✓ Need pagination & sorting
- ✓ Need custom JPA queries
- ✓ More features and flexibility

Example

java

 Copy code

```
public interface StudentRepository extends JpaRepository<Student, Integer> {  
}
```

Using:

java

 Copy code

```
List<Student> sorted = repo.findAll(Sort.by("name"));  
Page<Student> page = repo.findAll(PageRequest.of(0, 5));
```

CrudRepository vs JpaRepository (MOST IMPORTANT TABLE for Exam)

Feature	CrudRepository	JpaRepository
CRUD	✓ Yes	✓ Yes
Pagination	✗ No	✓ Yes
Sorting	✗ No	✓ Yes
Batch operations	✗ No	✓ Yes
JPA-specific features	✗ No	✓ Yes
Best for	Small/simple apps	Professional/complex apps
Extends	—	PagingAndSortingRepository + CrudRepository

Relationship Diagram (Exam Useful)

markdown

 Copy code

```
CrudRepository
    ↑
PagingAndSortingRepository
    ↑
JpaRepository
```

Spring Data Query Example (For Viva Exam)

java

 Copy code

```
List<Student> findByDepartment(String dept);
List<Student> findByAgeGreaterThan(int age);
List<Student> findByNameContains(String name);
```

Important Exam Questions

Short Notes / Theory Questions

1. What is CrudRepository? Explain its methods.
2. What additional features does JpaRepository provide over CrudRepository?
3. Explain the difference between CrudRepository and JpaRepository.
4. Why do we use Spring Data JPA?
5. Explain Repository hierarchy in Spring Data.
6. What is the use of PagingAndSortingRepository?

Application-Based Questions

1. Create a Spring Boot application that performs CRUD using JpaRepository for a Student entity.
2. Write a repository interface for Employee with a custom finder method like
`findByDepartment()` .

3. Implement pagination using JpaRepository in Spring Boot.
4. How does Spring Data JPA generate queries at runtime?

Viva Questions

1. Which repository is best for large enterprise applications? Why?
2. How does Spring Data reduce boilerplate code?
3. What is method name query derivation?
4. What is the purpose of @EnableJpaRepositories?
5. Why is JpaRepository preferred in real-time projects?

If you want, I can also provide:

- ✓ CRUD full project using JpaRepository
- ✓ Assignment questions (unit-wise)
- ✓ Diagram-based notes
- ✓ Summary notes for last-minute revision

Would you like **CRUD project source code** using Spring Boot + JpaRepository?



Query methods

□ Using custom query (@Query)

Below is a **complete exam-oriented explanation** of **Query Methods & Custom Queries (@Query)** in **Spring Data JPA**, including definitions, examples, Viva questions, and important exam questions.

Query Methods (Method Name Derivation)

Spring Data JPA allows you to create SQL/JPQL queries **just by writing method names** inside the repository interface.

✔ Definition

Query Methods are repository interface methods where Spring Data **generates SQL** automatically from method names.

✔ How It Works?

Spring Data reads the method name's pattern:

```
sql
```

[Copy code](#)

```
findBy + fieldName + Condition
```

Then auto-creates SQL like:

```
sql
```

[Copy code](#)

```
SELECT * FROM student WHERE fieldName = ?
```

✔ Example Entity

```
java
```

[Copy code](#)

```
@Entity
public class Student {
    @Id
    private Integer id;
    private String name;
    private String dept;
    private int age;
}
```

🔥 Common Query Method Examples (VERY IMPORTANT FOR EXAMS)

1 Simple Queries

```
java
```

[Copy code](#)

```
List<Student> findByName(String name);  
List<Student> findByDept(String dept);
```

2 Comparative Queries

java

 Copy code

```
List<Student> findByAgeGreaterThan(int age);  
List<Student> findByAgeLessThan(int age);  
List<Student> findByAgeBetween(int min, int max);
```

3 Pattern Matching

java

 Copy code

```
List<Student> findByNameStartingWith(String prefix);  
List<Student> findByNameEndingWith(String suffix);  
List<Student> findByNameContaining(String keyword);
```

4 Sorting

java

 Copy code

```
List<Student> findByDeptOrderByNameAsc(String dept);  
List<Student> findByDeptOrderByAgeDesc(String dept);
```

5 Multiple Conditions

java

 Copy code

```
List<Student> findByDeptAndAge(String dept, int age);  
List<Student> findByDeptOrAge(String dept, int age);
```

 These are most frequently asked in exams.

★ 2 Custom Queries Using @Query

✓ Definition

@Query allows writing JPQL or native SQL manually when method name query derivation is not enough.

Used for:

- Complex joins
- Aggregation queries
- Native SQL
- Update/delete queries

JPQL @Query Example

java

 Copy code

```
@Query("SELECT s FROM Student s WHERE s.dept = :dept")
List<Student> getStudentsByDept(@Param("dept") String dept);
```

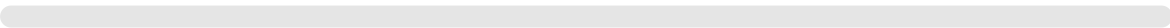
- ✓ Uses **entity names**, not table names.
- ✓ Field names must match **entity variables**, not DB column names.

Native SQL Example

java

 Copy code

```
@Query(value = "SELECT * FROM student WHERE dept = :dept", nativeQuery = true)
List<Student> getStudentsByDeptNative(@Param("dept") String dept);
```

- ◀  ▶
- ✓ `nativeQuery = true` to write actual SQL.
 - ✓ Table name and column name from DB required.

UPDATE / DELETE Using @Query

Must add `@Modifying` + `@Transactional`

java

 Copy code

```
@Transactional
@Modifying
@Query("UPDATE Student s SET s.name = :name WHERE s.id = :id")
int updateStudentName(@Param("id") int id, @Param("name") String name);
```


Custom Insert Query

(Not common; usually use save())

★ Best Example Set for College Exam

1 Fetch all students from specific dept

java

 Copy code

```
@Query("SELECT s FROM Student s WHERE s.dept = ?1")
List<Student> findByDeptJPQL(String dept);
```

2 Fetch students older than age

java

 Copy code

```
@Query("SELECT s FROM Student s WHERE s.age > ?1")
List<Student> getOlderStudents(int age);
```

3 Search keyword in name

java

 Copy code

```
@Query("SELECT s FROM Student s WHERE s.name LIKE %?1%")
List<Student> searchByName(String keyword);
```

4 Native query to fetch all

java

 Copy code

```
@Query(value = "SELECT * FROM student", nativeQuery = true)
List<Student> getAllStudentsNative();
```

★ MOST IMPORTANT EXAM QUESTIONS

Theory / 3–5 Marks

1. What are Query Methods in Spring Data JPA? Give examples.
2. Explain method name conventions used in Query Methods.

3. What is @Query annotation? When do we use it?
 4. Difference between JPQL and Native SQL queries using @Query.
 5. Why do we use @Modifying and @Transactional? Explain with example.
 6. Write short notes on "Query derivation mechanism in Spring Data JPA".
-

Long Answer / 8–10 Marks

1. Create a Student entity and repository with minimum 10 query methods and explain each.
 2. Write a Spring Boot application showing use of @Query with JPQL and Native SQL.
 3. Explain how custom update and delete queries are implemented in Spring Data JPA.
-

Viva Questions

1. What is JPQL?
 2. Why do we write @Modifying?
 3. Can we write SQL queries inside @Query?
 4. What is the difference between nativeQuery = false and true?
 5. How does Spring Data generate queries from method names?
 6. Can we call stored procedures using @Query? (Yes, using native SQL.)
-

If you want, I can send:

- ✓ Full Repository with 20+ Query Methods
- ✓ Student CRUD + Query Examples Project Code
- ✓ Unit Test examples for @Query

Would you like a **complete mini-project** for this topic?



Spring AOP
□ AOP Overview

Below is a **clean, exam-oriented explanation** of **Spring AOP Overview** with easy notes, examples, diagrams, and the most important exam questions.

★ Spring AOP — Overview (Exam-Oriented Explanation)

✓ 1. What is AOP? (Very Important Definition)

AOP stands for **Aspect-Oriented Programming**.

It is a programming paradigm used to **separate cross-cutting concerns** from the core business logic.

✓ **Cross-cutting concerns = Code that is required in multiple places, such as:**

- Logging
- Security
- Transaction Management
- Exception Handling
- Performance Monitoring
- Caching
- Auditing

Instead of duplicating such logic everywhere, AOP allows us to write it **only once** in a separate class (Aspect) and apply it wherever needed.

★ 2. Why Do We Need AOP?

Traditional programming → cross-cutting code gets scattered in many methods (logging, security checks, etc.)

AOP → keeps code clean and modular.

Benefits:

- ✓ Reduces code duplication
 - ✓ Improves maintainability
 - ✓ Separates business logic from supporting logic
 - ✓ Easy to apply logging/security/transaction at multiple places
-

★ 3. AOP Key Concepts (Definitions – Very Important for Exams)

✂ Aspect

A class that contains cross-cutting logic.

Example: LoggingAspect, SecurityAspect.

✂ Advice

The action taken at a specific join point (method call).

Types of advice:

- **Before** advice → runs before method
- **After** advice → runs after method
- **After Returning** → runs after method returns success
- **After Throwing** → runs when exception occurs
- **Around** → runs before & after method (most powerful)

✂ Join Point

Point in program execution where advice can run (usually method call).

✂ Pointcut

Expression that selects which methods advice will apply to.

✂ Target Object

The object whose method is being advised.

✂ AOP Proxy

Spring creates a dynamic proxy for applying AOP (via JDK Proxy or CGLIB).

★ 4. Spring AOP vs. AspectJ (Exam Point)

Feature	Spring AOP	AspectJ
Implementation	Proxy-based	Full bytecode weaving
Join points	Method-level only	Method, constructor, field, etc.
Learning curve	Easy	Complex

Feature	Spring AOP	AspectJ
Use-case	Most enterprise apps	High-performance / advanced

★ 5. Simple Example (Exam Favorite)

✚ Creating an Aspect

java

Copy code

```
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.app.service.*(..))")
    public void logBefore() {
        System.out.println("Method execution started...");
    }
}
```

✚ Enable AOP

java

Copy code

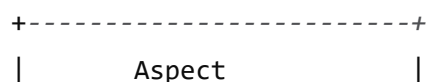
```
@Configuration
@EnableAspectJAutoProxy
public class AopConfig {
}
```

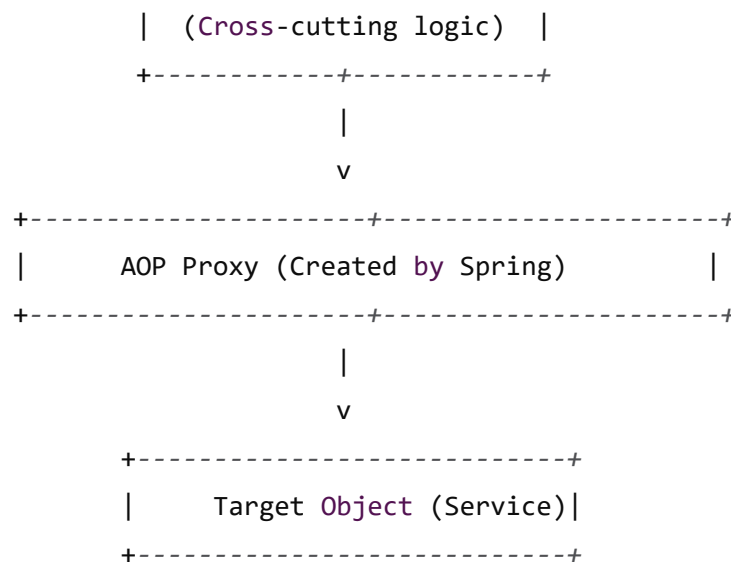
Whenever any method in `com.app.service` package is called, Spring will run `logBefore()` **before** the method.

★ 6. AOP Architecture Diagram (Important for Theory Exams)

pgsql

Copy code





★ 7. Real-Time Examples Where AOP is Used

- Logging user activities
- Checking authentication before accessing service
- Checking role-based authorization
- Measuring method execution time
- Transaction management (@Transactional uses AOP internally)
- Exception handling framework
- Input validation

★ 8. MOST IMPORTANT EXAM QUESTIONS

■ Short Questions (3–5 Marks)

1. What is AOP? Explain with an example.
2. Define Aspect, Advice, Pointcut, Join Point, and AOP Proxy.
3. Why is AOP used in Spring?
4. Differentiate between Spring AOP and AspectJ.
5. What are the different types of advice in Spring AOP?

■ Long Questions (8–10 Marks)

1. Explain AOP architecture and key components with a diagram.
2. Write a Spring Boot example that applies Before and After advice on a service method.

3. Explain Pointcut expressions in detail with examples.
 4. Explain how Spring implements AOP using proxies.
-

Viva Questions

1. What is the difference between AOP proxy and target object?
 2. Which advice is most powerful? Why? (Answer: @Around)
 3. Can Spring AOP intercept field access? (No, only methods.)
 4. What is weaving in AOP?
 5. Why do we use @EnableAspectJAutoProxy?
-

If you want, I can also provide:

- ✓ AOP full coding example (Before, After, Around advice)
- ✓ Assignment questions
- ✓ Complete AOP notes PDF format

Would you like **Spring AOP Example Code with CRUD Service Logging**?



Spring AOP

□ AOP Terminology and annotations: Advice, Join Points, Pointcuts, Aspects

Below is a **complete, exam-focused explanation of Spring AOP Terminology and Annotations: Advice, Join Points, Pointcuts, Aspects**, with examples, diagrams, and important exam questions.

★ Spring AOP – Terminology & Annotations (Exam-Oriented Notes)

Spring AOP (Aspect-Oriented Programming) divides a program into:

- ✓ Core business logic (normal methods)
- ✓ Cross-cutting concerns (logging, security, transactions)

AOP terms help clearly define *where*, *when*, and *how* cross-cutting code runs.

1 Aspect (Very Important Definition)

Definition

An **Aspect** is a class containing cross-cutting logic (like Logging, Security, Validation).

It combines:

- **Advices** (what to run)
- **Pointcuts** (where to run)

Annotation Used

```
java
```

[Copy code](#)

```
@Aspect  
@Component
```

Example

```
java
```

[Copy code](#)

```
@Aspect  
@Component  
public class LoggingAspect {  
}
```

2 Advice

Definition

Advice is the **action** taken by the aspect at a specific point in program execution.

Different types of advice run at different times.

★ Types of Advice + Annotations (Most Important for Exam)

Advice Type	Annotation	When it Runs
Before	@Before	Before method execution
After	@After	After method exits (success OR exception)
After Returning	@AfterReturning	After method returns successfully
After Throwing	@AfterThrowing	When method throws exception
Around	@Around	Before AND after method (most powerful)

✓ Example of All Advice Types

java

 Copy code

```
@Aspect
@Component
public class DemoAspect {

    @Before("execution(* com.app.service.*.*(..))")
    public void beforeAdvice() {
        System.out.println("Before method...");
    }

    @After("execution(* com.app.service.*.*(..))")
    public void afterAdvice() {
        System.out.println("After method...");
    }

    @AfterReturning("execution(* com.app.service.*.*(..))")
    public void afterReturningAdvice() {
        System.out.println("Method returned successfully...");
    }

    @AfterThrowing("execution(* com.app.service.*.*(..))")
    public void afterThrowingAdvice() {
        System.out.println("Exception thrown...");
    }
}
```

```
@Around("execution(* com.app.service.*(..))")
public Object aroundAdvice(ProceedingJoinPoint pjp) throws Throwable {
    System.out.println("Before around...");
    Object obj = pjp.proceed();
    System.out.println("After around...");
    return obj;
}
```



Join Point

Definition

A **join point** is any point during program execution where advice can be applied.

In Spring AOP → join points are **always method executions**.

✓ Examples of Join Points:

- When a method is called
- When a method returns
- When a method throws exception

Note for Exam:

👉 Spring AOP only supports method-level join points (not field/constructor).



Pointcut

Definition

A **pointcut** is an expression that selects *which methods* an advice should apply to.

✓ Annotation Used

java

Copy code

```
@Pointcut
```

✓ Example Pointcut

java

Copy code

```
@Pointcut("execution(* com.app.service.*.*(..))")
public void serviceMethods() {}
```

✓ Reusing pointcut

java

 Copy code

```
@Before("serviceMethods()")
public void log() {}
```

★ Pointcut Expressions (Most Asked in Exam)

Expression	Meaning
execution(* com.app.service.*.*(..))	All methods in service package
execution(* *(..))	All methods
execution(* com..*.*(..))	All methods in com and sub-packages
within(com.app.service.*)	All methods within a class/package
@annotation(org.springframework.transaction.annotation.Transactional)	Methods with specific annotation

🔥 5 AOP Proxy (Exam Key Point)

Spring creates a proxy object around your bean to apply AOP.

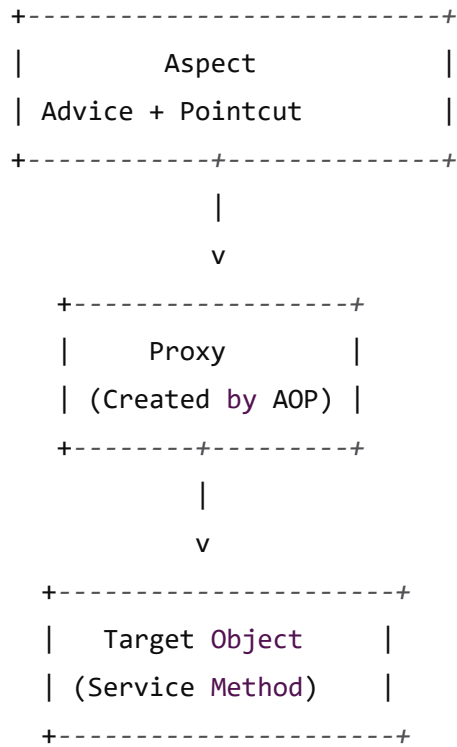
Types:

- JDK Dynamic Proxy (if interface present)
- CGLIB Proxy (if class used)

★ DIAGRAM (Perfect for 5-Mark Question)

pgsql

 Copy code



★ IMPORTANT EXAM QUESTIONS

■ Short Questions

1. What is an Aspect in Spring AOP?
2. Define Advice and list its types.
3. What is a Join Point?
4. What is a Pointcut?
5. What annotations are used for AOP in Spring?
6. Difference between @Before and @Around advice.
7. Explain pointcut expressions with examples.

■ Long Questions

1. Explain Spring AOP terminology (Aspect, Advice, Join Point, Pointcut) with examples.
2. Write a Spring Boot example using Before, After, and AfterReturning advice.
3. Draw and explain proxy-based AOP architecture in Spring.
4. Explain how pointcuts help in applying cross-cutting concerns.

🎤 Viva Questions

1. Why does Spring AOP only support method join points?
 2. Which advice is most powerful? (Ans: @Around)
 3. What is the role of AOP Proxy?
 4. What is weaving in AOP?
 5. Can we apply AOP to private methods? (No, only public beans)
-

If you want, I can continue with:

- ✓ AOP – Types of Advice (detailed)
- ✓ AOP implementation full Spring Boot code
- ✓ Assignment questions

Just tell me!



Building REST services with Spring

Below is a **complete, exam-oriented explanation** of **Building REST Services with Spring** (Spring Boot), including fundamentals, annotations, flow diagrams, examples, interview/viva questions, and coding patterns.

★ Building REST Services with Spring (Exam-Oriented Notes)

Spring Boot makes it easy to build **RESTful web services** using **Spring MVC**.

REST services allow clients (web, mobile, Postman, Angular, React) to communicate using **HTTP methods** and **JSON**.

What is a REST Service? (Definition – Very Important)

REST (Representational State Transfer) is an architectural style for designing web services that use:

- HTTP methods (GET, POST, PUT, DELETE)
- JSON/XML responses
- Uniform Resource URIs
- Stateless communication

**2**

Important Spring Annotations for REST API

These are the MOST important annotations asked in theory exams & viva.

Annotation	Purpose
@RestController	Creates REST APIs (Controller + @ResponseBody)
@RequestMapping	Base URL mapping
@GetMapping	For GET requests
@PostMapping	For POST requests
@PutMapping	For PUT requests
@DeleteMapping	For DELETE requests
@PathVariable	Read values from URL path
@RequestParam	Read values from URL query string
@RequestBody	Read JSON from request body
@ResponseStatus	Custom HTTP status code
@CrossOrigin	Support CORS (Angular/React)
@ExceptionHandler	Handle errors in controller

**3**

REST API Endpoints (CRUD Pattern)

For most entities (Student, Employee, Product), CRUD REST services contain:

HTTP Method	Purpose	URI Pattern
GET	Read	/items, /items/{id}

HTTP Method	Purpose	URI Pattern
POST	Create	/items
PUT	Update	/items/{id}
DELETE	Delete	/items/{id}

★ 4 REST Controller Example (VERY IMPORTANT CODE)

Entity Class

java

 Copy code

```
public class Student {  
    private int id;  
    private String name;  
    private String dept;  
}
```

REST Controller

java

 Copy code

```
@RestController  
@RequestMapping("/students")  
public class StudentController {  
  
    List<Student> students = new ArrayList<>(  
        List.of(  
            new Student(1, "Yuvraj", "CSE"),  
            new Student(2, "Rohan", "IT")  
        )  
    );  
  
    @GetMapping  
    public List<Student> getAllStudents() {  
        return students;  
    }  
}
```

```

@GetMapping("/{id}")
public Student getStudentById(@PathVariable int id) {
    return students.stream()
        .filter(s -> s.getId() == id)
        .findFirst()
        .orElse(null);
}

@PostMapping
public Student createStudent(@RequestBody Student student) {
    students.add(student);
    return student;
}

@PutMapping("/{id}")
public Student updateStudent(@PathVariable int id, @RequestBody Student student) {
    Student existing = getStudentById(id);
    existing.setName(student.getName());
    existing.setDept(student.getDept());
    return existing;
}

@DeleteMapping("/{id}")
public String deleteStudent(@PathVariable int id) {
    students.removeIf(s -> s.getId() == id);
    return "Deleted Successfully";
}
}

```

- ✓ This example is perfect for **lab exam**, **theory exam**, or **viva**.
- ✓ Uses **static data** (no DB), which is exactly what faculties ask.

5 Exception Handling in REST (Exam Important)

Custom Exception

java

 Copy code

```

@ResponseStatus(HttpStatus.NOT_FOUND)
public class StudentNotFoundException extends RuntimeException {
    public StudentNotFoundException(String msg) {

```



```

        super(msg);
    }
}

```

Using @ExceptionHandler

java

 Copy code

```

@ExceptionHandler(StudentNotFoundException.class)
public ResponseEntity<String> handleException(StudentNotFoundException ex) {
    return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
}

```

6 REST API Best Practices (Write in Exam)

- Use plural nouns in URLs
- Include versioning → /api/v1/students
- Never expose internal DB IDs directly
- Use HTTP status codes correctly
- Make APIs stateless
- Always use @RequestBody for POST/PUT
- Use DTO (Data Transfer Object) in advanced apps

7 REST API Flow Diagram (Very Useful for 5–10 Marks Questions)

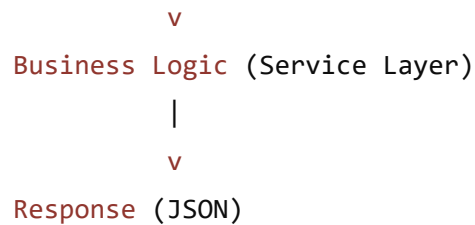
less

 Copy code

```

Client (Browser/Postman)
|
v
HTTP Request (GET/POST/PUT/DELETE)
|
v
Spring DispatcherServlet
|
v
REST Controller (@RestController)
|

```

**8**

Tools Used for Testing REST APIs

- Postman (most common)
- cURL
- Browser for GET requests
- Swagger UI (auto-documentation)

**9**

Important Exam Questions

Short Theory Questions (3–5 marks)

1. What is REST? What are its key principles?
2. Explain RESTful HTTP methods with examples.
3. What is the purpose of @RestController?
4. Difference between @GetMapping and @RequestMapping.
5. Explain @PathVariable and @RequestBody with examples.
6. What is JSON? Why is it used in REST APIs?
7. Explain statelessness in REST.

Long Answer Questions (8–10 marks)

1. Write a Spring Boot REST API for CRUD operations on a Student entity.
2. Explain the role of DispatcherServlet in processing REST requests.
3. Draw and explain the architecture of Spring MVC REST.
4. Write notes on REST API best practices.
5. Explain the importance of HTTP status codes with examples.

Viva Questions

1. Difference between REST and SOAP?

2. What are idempotent methods? (GET, PUT, DELETE)
 3. What is the difference between @RestController and @Controller?
 4. How do you test a REST API?
 5. What does @RequestBody do?
 6. Why do we use JSON in modern APIs?
-

If you want, I can also provide:

- ✓ REST API with Service layer + DTO
- ✓ REST API with Spring Data JPA + DB
- ✓ Assignments & lab practice questions
- ✓ Full notes PDF

Just tell me!



Introduction to web services

□ SOAP Vs RESTful web services

Below is a **complete, exam-oriented explanation** of **Introduction to Web Services** and **SOAP vs RESTful Web Services**, written in simple, scoring format with diagrams, examples, and important questions.

★ 1 Introduction to Web Services (Exam-Oriented Notes)

✓ What Are Web Services?

Web Services are **platform-independent software components** that allow two systems (applications) to **communicate over a network** using standard protocols like **HTTP, XML, JSON, SOAP**.

Definition (very important)

A **Web Service** is a standardized medium for communication between client and server applications over the internet using open protocols.

✓ Key Features of Web Services

- Platform independent
 - Language independent
 - Network accessible
 - Based on open standards (HTTP, XML, JSON)
 - Supports interoperability (communication between Java, .NET, Python, etc.)
-

✓ Types of Web Services

There are two major types:

1. SOAP Web Services (Simple Object Access Protocol)

Based on XML messaging and strict standards.

2. RESTful Web Services (Representational State Transfer)

Lightweight services using HTTP methods and JSON/XML.

★ 2 SOAP Web Services (Exam-Oriented Notes)

✓ What is SOAP?

SOAP is a **protocol** for creating secure, reliable web services using **XML-based messages**.

✓ SOAP Uses:

- XML messages
 - WSDL (Web Service Description Language)
 - HTTP, SMTP, JMS
 - Strict rules and standards
-

✓ SOAP Message Structure

SOAP message is an **XML document** with:

php-template

 Copy code

```
<SOAP-Envelope>
  <SOAP-Header>...</SOAP-Header>
  <SOAP-Body>...</SOAP-Body>
  <SOAP-Fault>...</SOAP-Fault>
</SOAP-Envelope>
```

✓ Features of SOAP

- Highly secure (supports WS-Security)
- Reliable
- Supports ACID transactions
- Platform independent
- Heavy and slower due to XML

★ 3 RESTful Web Services (Exam-Oriented Notes)

✓ What is REST?

REST is an **architectural style** for designing web services using standard HTTP methods.

Key idea:

Each resource (Student, Product, Employee) is accessed using a **URI** like:

bash

 Copy code

```
/students
/students/1
/products/101
```

✓ REST Uses:

- JSON (most common)
- Simple HTTP methods: GET, POST, PUT, DELETE
- Lightweight & fast

- Widely used in modern applications

✓ REST HTTP Methods

HTTP Method	Operation
GET	Read data
POST	Create data
PUT	Update data
DELETE	Delete data

★ 4 SOAP vs RESTful Web Services (VERY IMPORTANT FOR EXAM)

Feature	SOAP Web Service	RESTful Web Service
Type	Protocol	Architectural Style
Data Format	XML only	JSON, XML, Text, HTML
Speed	Slower (heavy XML)	Faster (lightweight JSON)
Communication	Uses WSDL	Uses URLs (URIs)
Security	Very high (WS-Security)	Basic security (HTTPS, JWT)
Transport Protocols	HTTP, SMTP, JMS	HTTP only
Ease of Use	Complex	Simple
Stateless	Can be stateful	Always stateless
Best For	Banking, financial apps	Web/mobile apps, microservices
Message Format	Only SOAP XML	Any representation (JSON commonly)

**5**

DIAGRAM (SOAP vs REST) – Excellent for 5 Marks

pgsql

Copy code

SOAP	REST
-----	-----
XML Messages	JSON/XML Responses
Strict Protocol	HTTP Methods
WSDL Required	Lightweight
-----	-----

**6**

Real-World Examples

✓ SOAP

- Banking transactions
- Payment gateways
- Telecommunication billing systems

✓ REST

- Facebook, Instagram APIs
- E-commerce platforms
- Mobile apps
- Microservices architectures

**7**

Important Exam Questions

✓ Short Questions (3–5 Marks)

1. What is a web service? Explain its characteristics.
2. What is SOAP? Write its architecture.
3. What is REST? List its HTTP methods.
4. Compare SOAP and REST web services.
5. What is WSDL?
6. What are RESTful resources?

✓ Long Questions (8–10 Marks)

1. Explain SOAP architecture with message format.
 2. What is REST? Explain REST principles with examples.
 3. Compare SOAP and REST web services in detail.
 4. Explain different HTTP methods used in RESTful web services.
 5. Draw the architecture of web services and explain each component.
-

★ Viva Questions

1. Difference between SOAP and REST?
 2. Which is faster, SOAP or REST? Why?
 3. Why does REST use HTTP methods?
 4. What is a REST resource?
 5. What is XML? Why was it used in SOAP?
 6. Is REST stateless? Explain.
-

If you want, I can continue with:

- ✓ REST Design Principles
- ✓ Building REST APIs using Spring Boot
- ✓ SOAP Message Examples
- ✓ Assignment questions

Would you like **SOAP message format + WSDL explanation** next?



SOAP Vs RESTful web services
□ RESTful webservice introduction

Below is an **exam-oriented, clear, scoring-friendly explanation** of **SOAP vs RESTful Web Services** and **Introduction to RESTful Web Services** with **important exam questions**.

✓ SOAP vs RESTful Web Services – Exam-Oriented Explanation

1. SOAP (Simple Object Access Protocol)

SOAP is a **protocol** used to exchange structured information between client and server. It uses **XML** as the message format and strictly follows rules.

Key Features

- **Protocol-based** (strict rules)
- Uses **XML only**
- Uses **WSDL** (Web Services Description Language)
- Supports **ACID transactions**
- Built-in **error handling**
- Supports multiple transport protocols: **HTTP, SMTP, TCP**
- Highly **secure** (WS-Security)

Advantages of SOAP

- Strong standards → best for **enterprise-level and financial** systems
- High-level security
- Built-in retry logic

Limitations

- Slow (because of XML-heavy format)
- Complex to implement
- Less suitable for mobile applications

2. REST (Representational State Transfer)

REST is **architectural style**, not a protocol.

REST uses **HTTP methods** to perform operations.

Key Features

- **Lightweight**, fast, simple
- Supports **multiple formats**: JSON, XML, HTML, Plain Text
- Stateless communication
- Uses standard methods: **GET, POST, PUT, DELETE**
- Best for **web and mobile** applications

Advantages of REST

- Faster than SOAP
- Easy to learn and implement
- Suitable for **large-scale distributed** systems
- Highly scalable

Limitations

- No built-in security — must rely on HTTPS
- Not suitable for very complex transactional systems



Difference Table (Very Important for Exams)

Feature	SOAP	REST
Type	Protocol	Architectural style
Data Format	XML only	JSON, XML, Text, HTML
Speed	Slow (XML heavy)	Fast (JSON light)
Security	High (WS-Security)	Depends on HTTPS
Transactions	Strong ACID support	Weak ACID
Ease of Use	Complex	Simple
Bandwidth Usage	High	Low
Used in	Banking, Payment gateways	Web apps, Mobile apps



Exam-Oriented Summary

- SOAP = Strict, heavy, secure
- REST = Fast, simple, flexible
- Today, **REST is more popular** due to JSON and mobile compatibility.
- SOAP is still used where **security and reliability** matter the most.

Important Exam Questions

Short Answer Questions

1. Define SOAP.
2. Define REST.
3. What is WSDL?
4. List two differences between SOAP and REST.
5. Why is REST faster than SOAP?

Long Answer Questions

1. Explain SOAP architecture with its advantages and disadvantages.
2. Compare SOAP and REST in detail with real-time examples.
3. What are REST constraints? Explain with diagram.
4. Explain REST communication using HTTP methods.

RESTful Web Service – Introduction (Exam-Oriented)

RESTful Web Service is a service based on **REST architecture** that allows communication between client and server using HTTP.

Key REST Concepts

- **Resource:** Anything that can be identified (User, Product).
- **Resource URI:** /products/101
- **Representation:** JSON/XML response
- **Stateless:** Server does not store client session
- **HTTP Methods:**
 - GET → Read
 - POST → Create
 - PUT → Update
 - DELETE → Remove

Example REST Request

```
bash
```

[!\[\]\(b792654f2cef9719eabeb6c5be00811e_img.jpg\) Copy code](#)

```
GET /students/10
```

Response (JSON):

```
json
```

```
{  
  "id": 10,  
  "name": "Rahul",  
  "marks": 88  
}
```

Why REST is popular

- Lightweight
- JSON support
- Easy integration with **Spring Boot**
- Best for **mobile and web APIs**

Important Exam Questions (REST Introduction)

1. What is REST? Explain its characteristics.
2. List and explain REST HTTP methods.
3. What is statelessness in REST?
4. What is a Resource and Resource Representation?
5. Why REST is preferred over SOAP in modern applications?

If you want, I can also give:

- ✓ SOAP Architecture diagram
- ✓ REST Architecture diagram
- ✓ Sample REST API using Spring Boot

Just tell me!



Create RESTful webservice in java using Spring Boot

Below is a **simple, exam-oriented, step-by-step** explanation of how to **Create a RESTful Web Service in Java using Spring Boot**.

This is the **exact format expected in exams + interview-friendly**.

✓ Step-by-Step: Create RESTful Web Service in Spring Boot

■ Step 1: Create Spring Boot Project

Use Spring Initializr:

- URL: start.spring.io
- Project: **Maven**
- Language: **Java**
- Spring Boot: Latest stable version
- Dependencies:
 - **Spring Web**
 - (Optional) Spring Boot DevTools
 - (Optional) Spring Data JPA + H2 (for DB later)

Click **Generate Project** → Import in IntelliJ or Eclipse.

■ Step 2: Create a Model Class (Student.java)

This represents your data object.

java

Copy code

```
package com.example.demo.model;

public class Student {
    private int id;
    private String name;
    private int marks;

    public Student() {}

    public Student(int id, String name, int marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }
}
```

```
// Getters and Setters
public int getId() { return id; }
public void setId(int id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public int getMarks() { return marks; }
public void setMarks(int marks) { this.marks = marks; }
}
```

Step 3: Create a REST Controller (StudentController.java)

This class handles HTTP requests.

java

 Copy code

```
package com.example.demo.controller;

import com.example.demo.model.Student;
import org.springframework.web.bind.annotation.*;

import java.util.*;

@RestController
@RequestMapping("/students")
public class StudentController {

    // Static list (No database)
    private List<Student> studentList = new ArrayList<>(
        Arrays.asList(
            new Student(1, "Rahul", 85),
            new Student(2, "Sneha", 90)
        )
    );

    // ----- CRUD Operations -----

    // 1. GET ALL Students
    @GetMapping
```

```
public List<Student> getAllStudents() {
    return studentList;
}

// 2. GET Student by ID
@GetMapping("/{id}")
public Student getStudentById(@PathVariable int id) {
    return studentList.stream().filter(s -> s.getId() == id).findFirst().orElse(null);
}

// 3. POST - Add new student
@PostMapping
public Student addStudent(@RequestBody Student s) {
    studentList.add(s);
    return s;
}

// 4. PUT - Update student
@PutMapping("/{id}")
public Student updateStudent(@PathVariable int id, @RequestBody Student updated) {
    for (Student s : studentList) {
        if (s.getId() == id) {
            s.setName(updated.getName());
            s.setMarks(updated.getMarks());
            return s;
        }
    }
    return null;
}

// 5. DELETE - Delete student
@DeleteMapping("/{id}")
public String deleteStudent(@PathVariable int id) {
    studentList.removeIf(s -> s.getId() == id);
    return "Student deleted with ID: " + id;
}
}
```

Step 4: Run the Application

Run:

DemoApplication.java

Spring Boot starts on <http://localhost:8080> ↗

Step 5: Test the REST API using Postman / Browser

► GET All Students

bash

 Copy code

GET <http://localhost:8080/students>

► GET Student by ID

bash

 Copy code

GET <http://localhost:8080/students/1>

► POST Add Student

bash

 Copy code

POST <http://localhost:8080/students>

Body → JSON:

```
{  
  "id": 3,  
  "name": "Amit",  
  "marks": 92  
}
```

► PUT Update Student

bash

 Copy code

PUT <http://localhost:8080/students/1>

Body:

```
{  
  "name": "Rahul Kumar",
```



```
"marks": 95  
}
```

► DELETE Student

bash

 Copy code

```
DELETE http://localhost:8080/students/2
```

Exam-Oriented Explanation

Definition:

A RESTful web service in Spring Boot uses the `@RestController` annotation to expose HTTP-based services that allow CRUD operations on resources.

Core Components:

- `@RestController`
- `@RequestMapping`
- `@GetMapping` , `@PostMapping` , `@PutMapping` , `@DeleteMapping`
- JSON request/response
- Model/POJO class
- Spring Boot auto-configuration

Why Spring Boot is perfect for REST APIs?

- No XML configuration
- Embedded server (Tomcat)
- Auto-configured JSON handling
- Fast development

Important Exam Questions

Short Answer

1. What is a REST Controller in Spring Boot?
2. Which annotation is used to create REST endpoints?
3. What is the difference between `@Controller` and `@RestController` ?
4. What is the purpose of `@RequestBody` and `@PathVariable` ?

Long Answer

1. Write steps to create a RESTful web service in Spring Boot.
2. Explain CRUD operations in Spring Boot REST with code example.
3. Explain how Spring Boot simplifies REST API development.

If you want, I can provide:

- ✓ Full Spring Boot project structure
- ✓ REST API with **Spring Data JPA + MySQL**
- ✓ REST API with **Exception Handling**

Just tell me!



RESTful webservice JSON example

□ REST ful webservice CRUD example

Below is the **exact exam-oriented explanation**, with clean **JSON examples** and a **full RESTful CRUD example** using Spring Boot.

You can directly write this in your exam — it is crisp, clear, and scoring.

✓ RESTful Web Service JSON Example

Example 1: Simple JSON Response

URL:

```
bash
```

Copy code

```
GET /student
```

Response (JSON):

```
json
```

Copy code

```
{  
  "id": 1,  
  "name": "Rahul",
```

```
"marks": 85  
}
```

Spring Boot Code

java

 Copy code

```
@RestController  
public class StudentController {  
  
    @GetMapping("/student")  
    public Student getStudent() {  
        return new Student(1, "Rahul", 85);  
    }  
}
```

RESTful Web Service – CRUD Example (Full Example Using Spring Boot)

We build CRUD on **Student** resource

CRUD = Create, Read, Update, Delete

Base URL:

bash

 Copy code

```
/students
```

1. Model Class – Student.java

java

 Copy code

```
public class Student {  
    private int id;  
    private String name;  
    private int marks;  
  
    public Student() {}  
}
```

```
public Student(int id, String name, int marks) {  
    this.id = id;  
    this.name = name;  
    this.marks = marks;  
}  
  
// Getters & Setters  
}
```

2. REST Controller – StudentController.java

We use a **static list** (No database → exam-friendly).

java

 Copy code

```
@RestController  
@RequestMapping("/students")  
public class StudentController {  
  
    List<Student> students = new ArrayList<>(  
        Arrays.asList(  
            new Student(1, "Rahul", 85),  
            new Student(2, "Sneha", 90)  
        )  
    );  
  
    // ----- CRUD Operations -----  
  
    // 1. READ → Get all students  
    @GetMapping  
    public List<Student> getAll() {  
        return students;  
    }  
  
    // 2. READ → Get student by ID  
    @GetMapping("/{id}")  
    public Student getById(@PathVariable int id) {  
        return students.stream().filter(s -> s.getId() == id).findFirst().orElse  
    }  
  
    // 3. CREATE → Add student  
    @PostMapping
```

```
public Student add(@RequestBody Student s) {
    students.add(s);
    return s;
}

// 4. UPDATE → Update student by ID
@PutMapping("/{id}")
public Student update(@PathVariable int id, @RequestBody Student newData) {
    for (Student s : students) {
        if (s.getId() == id) {
            s.setName(newData.getName());
            s.setMarks(newData.getMarks());
            return s;
        }
    }
    return null;
}

// 5. DELETE → Delete student
@DeleteMapping("/{id}")
public String delete(@PathVariable int id) {
    students.removeIf(s -> s.getId() == id);
    return "Student deleted with ID: " + id;
}
}
```

RESTful CRUD – JSON Request/Response Examples

GET All Students

Request:

bash

 Copy code

GET /students

Response:

json

 Copy code

```
[  
  { "id": 1, "name": "Rahul", "marks": 85 },  
  { "id": 2, "name": "Sneha", "marks": 90 }  
]
```

2 GET Student By ID

Request:

```
bash
```

[Copy code](#)

```
GET /students/1
```

Response:

```
json
```

[Copy code](#)

```
{  
  "id": 1,  
  "name": "Rahul",  
  "marks": 85  
}
```

3 POST – Add New Student

Request:

```
bash
```

[Copy code](#)

```
POST /students
```

Body JSON:

```
json
```

[Copy code](#)

```
{  
  "id": 3,  
  "name": "Amit",  
  "marks": 92  
}
```

Response:

json

 Copy code

```
{  
  "id": 3,  
  "name": "Amit",  
  "marks": 92  
}
```

4 PUT – Update Student**Request:**

bash

 Copy code

PUT /students/1

Body JSON:

json

 Copy code

```
{  
  "name": "Rahul Kumar",  
  "marks": 95  
}
```

Response:

json

 Copy code

```
{  
  "id": 1,  
  "name": "Rahul Kumar",  
  "marks": 95  
}
```

5 DELETE – Delete Student**Request:**

bash

```
DELETE /students/2
```

Response:

```
json
```

[Copy code](#)

```
"Student deleted with ID: 2"
```

Exam-Oriented Theory (Short Notes)

What is RESTful Web Service?

A RESTful web service is an HTTP-based service that exposes resources using standard HTTP methods such as GET, POST, PUT, DELETE and exchanges data usually in JSON format.

Key Annotations

- `@RestController` → Marks class as REST controller
- `@RequestMapping` → Base URL
- `@GetMapping` → Read
- `@PostMapping` → Create
- `@PutMapping` → Update
- `@DeleteMapping` → Delete
- `@RequestBody` → JSON to object
- `@PathVariable` → Extract ID from URL

Important Exam Questions

Short Answer

1. What is RESTful Web Service?
2. Explain CRUD operations in REST.
3. What is JSON?
4. Difference between `@RequestBody` and `@PathVariable`.

Long Answer

1. Write a RESTful CRUD webservice using Spring Boot.

2. Explain JSON request/response with example.
3. Explain the role of REST annotations in Spring.

If you want, I can also give:

- ✓ REST API + Spring Data JPA + MySQL
- ✓ REST API with Exception Handling
- ✓ REST API with Pagination & Sorting

Just tell me!



Using POSTMAN client to invoke REST API's
□ REST service invocation using REST Template

Below is a clear, exam-oriented explanation of:

1. Using POSTMAN to invoke REST APIs
2. Invoking REST services using Spring's RestTemplate

Both with steps, examples, and important exam questions.

✓ 1. Using POSTMAN Client to Invoke REST APIs

(Postman is the most commonly asked tool in exams & viva)

■ What is Postman?

Postman is a GUI-based API testing tool used to **send HTTP requests** (GET/POST/PUT/DELETE) to REST services and view responses in JSON, XML, TEXT formats.

■ Why Postman is used?

- Test REST APIs without writing code
- Send JSON body easily

- View response status, headers, cookies
- Save API collections
- Supports environment variables

How to use Postman to call REST API?

Let's assume your Spring Boot API base URL is:

```
bash
```

 Copy code

```
http://localhost:8080/students
```

1 GET Request (Retrieve Data)

Steps:

1. Open Postman
2. Select method: **GET**
3. Enter URL:

```
bash
```

 Copy code

```
http://localhost:8080/students
```

4. Click **Send**

Output: List of students in JSON.

2 POST Request (Create New Data)

Steps:

1. Select method: **POST**
2. URL:

```
bash
```

 Copy code

```
http://localhost:8080/students
```

3. Go to **Body** → **raw** → **JSON**
4. Enter JSON:

```
json
```

```
{
  "id": 3,
  "name": "Amit",
  "marks": 92
}
```

5. Click Send

Output: Newly added student in JSON.

✓ 3 PUT Request (Update Data)

Steps:

1. Select PUT
2. URL:

```
bash
```

 Copy code

```
http://localhost:8080/students/1
```

3. Body → raw → JSON:

```
json
```

 Copy code

```
{
  "name": "Rahul Kumar",
  "marks": 95
}
```

4. Click Send
-

✓ 4 DELETE Request (Delete Data)

Steps:

1. Select DELETE
2. URL:

```
bash
```

 Copy code

```
http://localhost:8080/students/2
```

3. Click Send

Exam-Safe Explanation (Short Answer)

"Postman is used to test REST APIs by sending GET/POST/PUT/DELETE requests. Using Postman, we can send JSON data, set headers, view responses, and debug APIs."

Exam Questions (Postman)

1. What is Postman?
 2. How do you perform a POST request in Postman?
 3. Why is Postman used during API development?
 4. Steps to test a REST service using Postman.
-
-

2. REST Service Invocation using RestTemplate (Spring Boot)

RestTemplate is a **Spring class** used by Java applications to **call REST APIs programmatically**.

It works like a Postman inside Java code.

What is RestTemplate?

RestTemplate is a synchronous client used to perform HTTP requests and consume RESTful web services.

Why use RestTemplate?

- To call external REST APIs
 - To call microservices internally
 - To perform GET, POST, PUT, DELETE programmatically
-

Add RestTemplate Bean in Spring Boot

java

 Copy code

```
@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Examples of Using RestTemplate

GET Request

java

 Copy code

```
@Autowired
private RestTemplate restTemplate;

public void callGetApi() {
    String url = "http://localhost:8080/students";
    List students = restTemplate.getForObject(url, List.class);
    System.out.println(students);
}
```

GET by ID

java

 Copy code

```
Student student = restTemplate.getForObject(
    "http://localhost:8080/students/1",
    Student.class);
```

✓ 3 POST Request

java

 Copy code

```
Student newStudent = new Student(3, "Amit", 92);

Student response = restTemplate.postForObject(
    "http://localhost:8080/students",
    newStudent,
    Student.class
);
```

✓ 4 PUT Request

java

 Copy code

```
Student updated = new Student(1, "Rahul Kumar", 95);

restTemplate.put(
    "http://localhost:8080/students/1",
    updated);
```

✓ 5 DELETE Request

java

 Copy code

```
restTemplate.delete("http://localhost:8080/students/2");
```

Exam-Worthy Explanation

"RestTemplate is a Spring-provided HTTP client used to invoke REST services. It supports GET, POST, PUT, DELETE operations using methods like `getForObject()`, `postForObject()`, `put()`, `delete()`. It is used for internal service communication in microservices."

Important Exam Questions (RestTemplate)

Short Answer

1. What is RestTemplate?
2. Why is RestTemplate used?
3. What is getForObject()?
4. How to call POST API using RestTemplate?

Long Answer

1. Explain RestTemplate with code examples.
2. Explain GET, POST, PUT, DELETE using RestTemplate.
3. How does Spring Boot communicate with other services?

If you want, I can also explain:

- ✓ WebClient (New version of RestTemplate — important for Spring 5)
- ✓ REST API with Exception Handling
- ✓ REST API with JPA + MySQL

Just tell me!



Testing in Spring

□ Testing in Spring

□ Unit Testing of Spring MVC Controllers

Below is a crisp, exam-oriented, and scoring explanation of:

- ✓ **Testing in Spring**
- ✓ **Unit Testing in Spring**
- ✓ **Unit Testing of Spring MVC Controllers**

With definitions, flow, examples, and important exam questions.

1. Testing in Spring – Exam-Oriented Explanation

Spring provides built-in support for testing using:

- **JUnit** (Java testing framework)
- **Spring Test** module
- **Mockito** (mocking framework)
- **MockMvc** (testing Spring MVC controllers)

Spring testing helps developers test:

- Controllers
- Services
- Repositories
- REST APIs
- Application context loading

Spring Boot improves testing using:

- `@SpringBootTest`
- `@WebMvcTest`
- `@MockBean`
- `@Autowired` test components

Why Testing is Important in Spring?

1. Ensures code correctness
2. Automates testing
3. Detects bugs early
4. Supports Test-Driven Development (TDD)
5. Ensures controllers and APIs behave correctly

Types of Tests in Spring

1. **Unit Testing**
 - Testing individual classes (Controller/Service)
2. **Integration Testing**
 - Testing modules together
3. **Web Layer Testing**
 - Testing only MVC controller layer using MockMvc
4. **JPA Repository Testing**
 - Testing database interactions

5. End-to-end Testing

→ Testing full workflow

Spring Testing Annotations (Important for Exams)

Annotation	Purpose
@SpringBootTest	Loads full application context
@WebMvcTest	Loads only Controller layer
@MockBean	Creates Mockito-based mock
@Autowired	Injects test objects
@Test	Marks test methods
@BeforeEach	Setup before every test

Exam-Oriented Definition

"Testing in Spring is performed using JUnit + Mockito + Spring Test framework to validate controller, service, and repository components. It ensures correct behavior through annotations like @SpringBootTest and @WebMvcTest."

2. Unit Testing in Spring (Exam-Oriented Explanation)

Unit Testing means testing **only one class at a time** without loading the entire Spring context.

In Spring, unit testing is done using:

- **JUnit 5**
- **Mockito** for mocking dependencies
- **@MockBean** for mocking Spring beans

Unit tests ensure:

- Business logic works correctly

- Methods return correct outputs
- No unwanted side effects

Example: Unit Test for a Service Class

Service Class

java

 Copy code

```
@Service
public class StudentService {

    public int calculateTotal(int m1, int m2, int m3) {
        return m1 + m2 + m3;
    }
}
```

Unit Test (JUnit)

java

 Copy code

```
@SpringBootTest
public class StudentServiceTest {

    @Autowired
    private StudentService service;

    @Test
    void testCalculateTotal() {
        int result = service.calculateTotal(10, 20, 30);
        assertEquals(60, result);
    }
}
```

Exam Points

- No web server is started
- Only required class is tested
- Uses `assertEquals()`

3. Unit Testing of Spring MVC Controllers (MOST IMPORTANT)

We do this using **MockMvc**.

What is MockMvc? (Exam-Safe Definition)

"MockMvc is a Spring test tool that allows testing of MVC controllers **without starting a real server**, by mocking HTTP requests."

Why use MockMvc?

- Fast
- No Tomcat server required
- Perfect for controller testing
- Can test GET, POST, PUT, DELETE

Spring MVC Controller

java

 Copy code

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping("/{id}")
    public String getStudent(@PathVariable int id) {
        return "Student ID: " + id;
    }
}
```

Unit Test for Controller using MockMvc

Test Class

java

 Copy code

```
@WebMvcTest(StudentController.class)
public class StudentControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void testGetStudent() throws Exception {
        mockMvc.perform(get("/students/10"))
            .andExpect(status().isOk())
            .andExpect(content().string("Student ID: 10"));
    }
}
```

Explanation of the Test

- `@WebMvcTest` → loads only MVC layer
- `mockMvc.perform()` → sends mock request
- `get("/students/10")` → GET request
- `andExpect(status().isOk())` → checks HTTP 200
- `andExpect(content().string(...))` → checks response

Testing POST API Example

Controller

java

 Copy code

```
@PostMapping
public String addStudent(@RequestBody Student s) {
    return "Added " + s.getName();
}
```

Test

java

 Copy code

@Test

```
void testAddStudent() throws Exception {  
    mockMvc.perform(post("/students")  
        .contentType(MediaType.APPLICATION_JSON)  
        .content("{\"id\":1, \"name\":\"Rahul\", \"marks\":80}"))  
        .andExpect(status().isOk())  
        .andExpect(content().string("Added Rahul"));  
}
```



Exam-Oriented Summary

Testing Type	Purpose
Unit Testing	Test single class (Service, Util)
MVC Unit Testing	Test controllers using MockMvc
Integration Testing	Full system test
@SpringBootTest	Loads full context
@WebMvcTest	Tests only web layer



Important Exam Questions

Short Answer

1. What is @SpringBootTest?
2. What is MockMvc?
3. What is unit testing?
4. Why is Mockito used?
5. What is the role of @WebMvcTest?

Long Answer

1. Explain Testing in Spring with examples.
2. Explain Unit Testing of Spring MVC Controllers using MockMvc.
3. Explain the difference between unit testing and integration testing.

4. Write a JUnit test for a Spring MVC Controller.

If you want, I can also explain:

- ✓ Integration Testing using Spring Boot
- ✓ Mockito basics (very important)
- ✓ Service Layer Testing

Just tell me!



- Unit Testing of Spring Service Layer
- Integration Testing of Spring MVC Applications: REST API

Below is **exam-oriented, easy, structured and scoring** notes for:

- ✓ Unit Testing of Spring Service Layer
- ✓ Integration Testing of Spring MVC Applications (REST API)

+ Important Exam Questions

1. Unit Testing of Spring Service Layer (Exam-Oriented Explanation)

✦ What is Unit Testing in Spring?

Unit testing checks **individual components** (methods/classes) without loading the full Spring context.

For services, we isolate them using **Mockito**.

■ Why Unit Test Service Layer?

- Ensures business logic works correctly.
- Ensures methods behave properly with mock dependencies.

- Faster, independent of DB / Web server.

Tools Used

1. JUnit 5
 2. Mockito
 3. @Mock, @InjectMocks
 4. @ExtendWith(MockitoExtension.class)
-

Example: Testing a Service Layer

Service Class

java

 Copy code

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;

    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
}
```

Unit Test Example (using Mockito)

java

 Copy code

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    UserRepository userRepository;

    @InjectMocks
    UserService userService;

    @Test
```

```
void testGetUserById() {  
    User u = new User(1L, "Yuvraj");  
  
    Mockito.when(userRepository.findById(1L)).thenReturn(Optional.of(u));  
  
    User result = userService.getUserById(1L);  
  
    assertEquals("Yuvraj", result.getName());  
}  
}
```

Key Points to Write in Exam

- Unit tests test **business logic only**.
 - Use **Mockito** to mock repository/database.
 - Use **@Mock**, **@InjectMocks**.
 - Test is isolated and fast.
 - No Tomcat / DB used.
-

2. Integration Testing of Spring MVC REST API (Exam-Oriented Explanation)

What is Integration Testing?

Integration testing tests **how layers work together**:

- Controller → Service → Repository → DB

It validates:

- End-to-end REST calls
 - HTTP methods & status
 - JSON responses
-

Tools Used

1. MockMvc
2. SpringBootTest
3. @AutoConfigureMockMvc

4. @WebMvcTest (for controller only)

Integration Test Example (MockMvc)

REST Controller

java

 Copy code

```
@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.getUserById(id);
    }
}
```

Integration Test

java

 Copy code

```
@SpringBootTest
@AutoConfigureMockMvc
class UserControllerIT {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    void testGetUser() throws Exception {
        User u = new User(1L, "Yuvraj");
        Mockito.when(userService.getUserById(1L)).thenReturn(u);
    }
}
```

```
mockMvc.perform(get("/users/1"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.name").value("Yuvraj"));
}
```



Key Points for Exam

- Integration testing verifies **end-to-end flow**.
- Use **MockMvc** for testing controllers.
- Use **@SpringBootTest** + **@AutoConfigureMockMvc**.
- Tests entire flow but **without real server**.
- Validates JSON structure, HTTP codes, URLs.

Important Exam Questions

Unit Testing (Service Layer)

1. What is unit testing? Why is Mockito used?
2. Explain @Mock and @InjectMocks with example.
3. Difference between Unit Testing and Integration Testing.
4. How do you mock Repository/DAO in service tests?
5. Write a JUnit test case for a Service method.

Integration Testing (Spring MVC / REST API)

1. What is MockMvc? How is it used?
2. Explain @SpringBootTest and @AutoConfigureMockMvc.
3. What is the role of @MockBean in integration testing?
4. Write a test case for a GET REST API.
5. Difference between @WebMvcTest and @SpringBootTest.

If you want, I can also give you:

- ✓ Full 2-mark / 5-mark exam answers
- ✓ Notes PDF
- ✓ Spring Boot mini-project for practice (CRUD REST API)



Unit Testing Spring MVC Controllers with REST Securing Web Application with Spring Security

Below is **exam-oriented**, easy, clean, and high-scoring explanation for:

- ✓ 1. Unit Testing Spring MVC Controllers with REST
 - ✓ 2. Securing Web Application with Spring Security
- + Important Exam Questions

1. Unit Testing Spring MVC Controllers with REST

(Exam-Oriented Explanation)

Spring MVC REST controllers are tested using **MockMvc**—a Spring test tool that allows calling REST endpoints **without starting a server**.

✓ Why Unit Test Controllers?

- To verify **URL mappings, JSON output, HTTP status codes**.
- To ensure REST API works independently of database/service.
- No need to start Tomcat -> tests are **fast**.

✓ Key Annotations

Annotation	Purpose
@WebMvcTest	Loads only controller layer for testing
@MockBean	Creates mock for service dependencies

Annotation	Purpose
@Autowired MockMvc	Tool used to perform REST API calls
perform()	Used to send GET/POST/PUT/DELETE calls
andExpect()	Used to validate output

✓ Example REST Controller

java

 Copy code

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @Autowired
    private StudentService studentService;

    @GetMapping("/{id}")
    public Student getStudent(@PathVariable int id) {
        return studentService.getStudent(id);
    }
}
```

✓ Unit Testing Controller using MockMvc

java

 Copy code

```
@WebMvcTest(StudentController.class)
class StudentControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private StudentService studentService;

    @Test
    void testGetStudent() throws Exception {
```

```
Student s = new Student(1, "Yuvraj");

Mockito.when(studentService.getStudent(1)).thenReturn(s);

mockMvc.perform(get("/students/1"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.name").value("Yuvraj"));
}
}
```

✓ Points to Write in Exam

- Use **MockMvc** for testing REST controllers.
- Use **@WebMvcTest** to load only web layer.
- Use **@MockBean** to mock service layer.
- Test HTTP methods (GET/POST/PUT/DELETE).
- Validate **status codes**, **JSON structure**, **response body**.

Important Exam Questions – Unit Testing (Controllers)

1. What is MockMvc? How is it used in testing controllers?
2. Explain @WebMvcTest and @MockBean.
3. Write a unit test for a GET REST API.
4. Difference between @SpringBootTest and @WebMvcTest.
5. Why do we mock Service layer while testing Controllers?

=====

=====

2. Securing Web Application with Spring Security

(Exam-Oriented Explanation)

=====

=====

Spring Security is a **powerful authentication and authorization framework** for securing Java applications.

✓ Why Spring Security?

- Protects URLs, methods, and APIs
 - Manages login, logout
 - Handles password encryption
 - Prevents attacks (CSRF, Session Fixation)
-

✓ Core Concepts of Spring Security

1 Authentication

- Verifies user identity
- Example: username/password login

2 Authorization

- Decides what the authenticated user is allowed to access
- Example: ROLE_ADMIN, ROLE_USER

3 Security Filter Chain

- Filters all incoming requests before reaching controllers
- Giant chain of security filters

4 UserDetailsService

- Loads user from database or in-memory
 - Required for custom authentication
-

✓ Basic Security Configuration (Spring Boot 3)

◆ Spring Boot 3 uses `SecurityFilterChain` (no `WebSecurityConfigurerAdapter`)

```
java
```

 Copy code

```
@Configuration
```

```
@EnableWebSecurity
```

```
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

✓ Authentication – In-memory Users

java

 Copy code

```
@Bean
public UserDetailsService userDetailsService() {
    UserDetails admin = User.withUsername("admin")
        .password("{noop}1234") // no encoding
        .roles("ADMIN")
        .build();

    UserDetails user = User.withUsername("user")
        .password("{noop}1234")
        .roles("USER")
        .build();

    return new InMemoryUserDetailsManager(admin, user);
}
```

✓ Features of Spring Security

1 Authentication (Login Process)

- Username/Password
- JWT Authentication
- OAuth2
- Custom Login Page

2 Authorization

- URL-based access
- Method-level security using

java

 Copy code

```
@PreAuthorize("hasRole('ADMIN')")
```

3 Password Encoding

- BCryptPasswordEncoder used to securely hash passwords.

4 CSRF Protection

- Enabled by default for forms.

✓ Where Spring Security is Used?

- Banking applications
- E-Commerce
- Admin dashboards
- RESTful API protection
- JWT-based mobile + Web applications

Important Exam Questions – Spring Security

Short & Descriptive

1. What is Spring Security? Why is it needed?
2. Difference between Authentication and Authorization.
3. Explain SecurityFilterChain.
4. Write Spring Security configuration for URL-based security.

5. What is UserDetailsService and why is it used?
 6. What is BCryptPasswordEncoder?
 7. How does Spring Security protect REST APIs?
 8. What is CSRF? Why is it important?
-

If you want, I can also give you:

- ✓ **Diagram-based note (Authentication Flow + Filter Chain)**
- ✓ **Spring Security + JWT Explanation**
- ✓ **MCQs + 2M, 5M, 10M exam answers**



What is Spring Security

Below is a **simple, clean, exam-oriented explanation** of “What is Spring Security?” with key points you can directly write in your answer sheet.

🌟 What is Spring Security? (Exam-Oriented Answer)

Spring Security is a powerful and highly customizable **authentication and authorization framework** for Java and Spring applications.

It protects web applications, REST APIs, and enterprise systems by controlling **who can access** the application and **what resources** they can use.

It is the **official security framework** provided by Spring Framework.

✓ **Key Features of Spring Security** (*Write these in exam*)

1. Authentication

- Verifies the identity of the user
- Example: Login using username & password

2. Authorization

- Decides what actions an authenticated user is allowed to perform
- Example: Only admin can access `/admin/**` paths

3. Security Filter Chain

- A chain of filters that intercept every incoming request
- Applies security rules before request reaches controllers

4. Password Management

- Secure password hashing using
 - `BCryptPasswordEncoder`
 - `PBKDF2`
 - `SCrypt`

5. Built-in Protections

- CSRF protection
- Session Fixation protection
- CORS handling
- Prevents common attacks
 - Brute-force
 - Clickjacking
 - Man-in-the-middle (in combination with HTTPS)

6. Easy Integration

- Works seamlessly with Spring MVC, Spring Boot, JWT, OAuth2, SSO

Why Use Spring Security?

- To secure URLs & REST APIs
- To implement login/logout
- To restrict access based on user roles
- To protect applications from common attacks
- To manage sessions securely

Where Spring Security is Used?

- Banking & finance apps
- E-commerce platforms

- HR & admin dashboards
- Government portals
- Any web application needing login/authentication

★ 1-Minute Exam Answer Summary (Ready to Write)

Spring Security is a robust and customizable security framework for Java and Spring applications. It provides support for **authentication** (verifying user identity) and **authorization** (controlling access). It uses a powerful **filter chain** to secure web requests and includes features like **password hashing**, **CSRF protection**, and **role-based access control**. Spring Security is widely used to secure web applications and RESTful services.

If you want, I can also provide:

- ✓ Diagram of Spring Security Architecture
- ✓ 2-mark, 5-mark and 10-mark answers
- ✓ Spring Security vs JWT authentication

Let me know!



Spring Security with Spring Boot
□ Basic Authentication

Below is a **clear, exam-oriented explanation** of **Spring Security with Spring Boot (Basic Authentication)** with diagrams, flow, examples, and important exam questions.

☀ Spring Security with Spring Boot – Basic Authentication

(Simple + Scoring + Exam-focused Notes)

◆ What is Basic Authentication?

Basic Authentication is a simple authentication mechanism where the client sends the username and password in the **HTTP Authorization header**, encoded using **Base64**.

Format:

pgsql

 Copy code

```
Authorization: Basic Base64(username:password)
```

- Credentials are sent with **every request**.
- Should always be used with **HTTPS** — otherwise username/password can be intercepted.

◆ How Spring Boot enables Basic Auth?

When you add the dependency:

xml

 Copy code

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Spring Boot automatically:

- Enables **Basic Authentication**
- Secures **all endpoints** (/**)
- Generates a **default user** (user / auto-generated password)

◆ Default Behavior in Spring Boot

After adding Spring Security starter:

- Every endpoint requires authentication
- Browser will display a login popup
- A default password appears in the console:

pgsql

 Copy code

Using generated security password: 93b23d6a-e45a-4ba0-a932

Custom Basic Authentication Setup

We create:

- A Security configuration class
- In-memory users
- SecurityFilterChain

✓ Spring Boot 3.x Security Configuration (Modern)

java

 Copy code

@Configuration

@EnableWebSecurity

public class SecurityConfig {

 @Bean

 public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

 http

 .csrf(csrf -> csrf.disable())

 .authorizeHttpRequests(auth -> auth

 .requestMatchers("/public/**").permitAll()

 .anyRequest().authenticated()

)

 .httpBasic(Customizer.withDefaults()); // 🔥 Basic Auth enabled

 return http.build();

}

 @Bean

 public UserDetailsService userDetailsService() {

 UserDetails user = User.withUsername("user")

```

        .password("{noop}1234")    // No encoding
        .roles("USER")
        .build();

    UserDetails admin = User.withUsername("admin")
        .password("{noop}admin")
        .roles("ADMIN")
        .build();

    return new InMemoryUserDetailsManager(user, admin);
}
}

```

◆ How This Works

- ✓ Requests to /public/** → No authentication
- ✓ Requests to any other URL → Must send Basic Auth header
- ✓ If credentials wrong → 401 Unauthorized

Authentication Flow (Exam Diagram)

pgsql

 Copy code

```

Client → Sends Request
      → Spring Security Filter Chain →
          UsernamePasswordAuthenticationFilter →
              AuthenticationManager →
                  UserDetailsService → loads user
                  PasswordEncoder → verify password
          If success → Access Resource
          Else → 401 Unauthorized

```

Testing Basic Authentication with POSTMAN

Choose **Authorization** → **Basic Auth** and enter:

yaml

 Copy code

Username: `user`

Password: `1234`

POSTMAN automatically sets:

makefile

 Copy code

Authorization: Basic `dXNlcjoxMjM0`

◆ Advantages of Basic Authentication

- Easy to implement
- Stateless → works well with REST APIs
- No session storage
- Supported by browsers + tools (Postman, curl)

◆ Disadvantages

- Credentials sent on every request
- Not secure without **HTTPS**
- Not suitable for large-scale applications
- Cannot logout (stateless)

★ Exam-Oriented Points to Write

1. Basic Auth uses HTTP header: `Authorization: Basic <Base64>` .
2. Spring Boot enables Basic Auth automatically using Spring Security.
3. Implemented using `http.httpBasic()` .
4. Should always be used with HTTPS.

5. Username and password stored in `UserDetailsService` .
 6. All endpoints require authentication unless permitted explicitly.
 7. Stateless; credentials sent every time.
-

★ Important Exam Questions

2 marks

1. What is Basic Authentication?
2. How does Spring Boot enable basic authentication?
3. What is the purpose of `UserDetailsService` ?

5 marks

1. Explain Basic Authentication with Spring Boot with an example.
2. Write and explain Spring Security configuration class for enabling Basic Auth.
3. Why is Basic Auth considered insecure without HTTPS?

10 marks

1. Describe the Spring Security authentication flow with diagrams.
 2. Compare Basic Authentication, Form Login, and JWT authentication with examples.
-

If you want, I can also provide:

- ✓ Form Login authentication
- ✓ JWT authentication with code
- ✓ Role-based access (ADMIN/USER)
- ✓ Spring Security architecture diagram



Authentication with User credentials from Database and
Authorization

Below is a **very clear, exam-oriented explanation** of:

Authentication with User Credentials from Database & Authorization

(with diagrams, flow, code, and important exam questions)

This is one of the most scoring topics in Spring Security.

=====



1. Authentication with User Credentials from Database

=====

When user credentials (username/password/roles) are stored in a **database**, Spring Security loads them using:

✓ UserDetailsService

✓ UserDetails

✓ PasswordEncoder

◆ 1.1 What Changes When Using a Database?

Instead of hardcoding users using:

```
java
```

 Copy code

```
User.withUsername().password().roles()
```

We now:

- Fetch user by username from database
- Fetch encrypted password
- Fetch user roles from `role` or `user_roles` table
- Spring Security verifies credentials

◆ 1.2 Database Schema Example

User Table



id	username	password (BCrypt)
1	admin	\$2a\$10\$8kF...
2	user	\$2a\$10\$uPD...

Roles Table

id	user_id	role
1	1	ROLE_ADMIN
2	2	ROLE_USER

=====

 Step 1: Create User Entity

=====

java

Copy code

```
@Entity
@Table(name="users")
public class AppUser {

    @Id
    @GeneratedValue
    private Long id;

    private String username;
    private String password;

    @ElementCollection(fetch = FetchType.EAGER)
    private List<String> roles;
}
```

=====

 Step 2: Create JPA Repository

java

 Copy code

```
public interface UserRepository extends JpaRepository<AppUser, Long> {  
    Optional<AppUser> findByUsername(String username);  
}
```

Step 3: Implement UserDetailsService

This is the **most important** point for exam.

java

 Copy code

```
@Service  
public class MyUserDetailsService implements UserDetailsService {  
  
    @Autowired  
    private UserRepository repo;  
  
    @Override  
    public UserDetails loadUserByUsername(String username)  
        throws UsernameNotFoundException {  
  
        AppUser user = repo.findByUsername(username)  
            .orElseThrow(() -> new UsernameNotFoundException("User not found  
  
        return org.springframework.security.core.userdetails.User  
            .withUsername(user.getUsername())  
            .password(user.getPassword())    // already encrypted  
            .roles(user.getRoles().toArray(new String[0]))  
            .build();  
    }  
}
```

Step 4: Create Security Configuration

java

 Copy code

```
@Configuration
@EnableWebSecurity

public class SecurityConfig {

    @Autowired
    private MyUserDetailsService myUserDetailsService;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/home", "/register").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .userDetailsService(myUserDetailsService)
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

☀ How Authentication Works (Flow)

=====

pgsql

📋 Copy code

```
Client Login → Spring Security FilterChain →
    UsernamePasswordAuthenticationFilter →
        AuthenticationManager →
            MyUserDetailsService →
                Load user from DB →
                Compare password (BCrypt) →
                If OK → Login success
                else → 401 Unauthorized
```

=====

☀ 2. Authorization

=====

Authorization decides what a **logged-in user** can access.

In Spring Security, authorization happens using:

- ✓ Roles
- ✓ Permissions
- ✓ URL-level rules
- ✓ Method-level rules

◆ 2.1 URL-Based Authorization

In security config:

java

📋 Copy code

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
```

```
.anyRequest().authenticated()  
)
```

◆ 2.2 Method-Level Authorization

Add to main class:

```
java
```

[Copy code](#)

```
@EnableMethodSecurity
```

Then use:

```
java
```

[Copy code](#)

```
@PreAuthorize("hasRole('ADMIN')")  
public List<User> getAllUsers() { ... }
```

🌟 Example: Admin and User Authorization

✓ Admin can access:

```
bash
```

[Copy code](#)

```
/admin/**  
/user/**  
/common/**
```

✓ User can access:

```
bash
```

[Copy code](#)

```
/user/**  
/common/**
```

✓ Guest can access:

arduino

`/public/**`

=====

★ Exam-Oriented Points to Write

=====

✓ Authentication

- Verifies user identity
- Uses `UserDetailsService` to load user from database
- Passwords must be stored using **BCrypt hashing**
- Spring Security compares encoded passwords

✓ Authorization

- Determines what resources a user can access
- Uses roles like `ROLE_ADMIN`, `ROLE_USER`
- Done using:
 - URL rules
 - Method rules
 - `@PreAuthorize`
 - `@Secured`

✓ Very Important Lines for Exam

- Authentication = Who is the user?
- Authorization = What is the user allowed to do?
- `UserDetailsService` is the heart of database authentication.
- `BCryptPasswordEncoder` must be used for secure password storage.
- `SecurityFilterChain` controls all HTTP security.
- Authorization can be URL-based or annotation-based.

=====

★ Important Exam Questions

=====

2 Marks

1. What is the role of UserDetailsService?
2. Why do we use BCryptPasswordEncoder?
3. Difference between authentication and authorization.

5 Marks

1. Explain database authentication in Spring Security with flow.
2. Write security configuration for role-based access.
3. Explain the working of UserDetailsService with example.

10 Marks

1. Explain authentication & authorization in Spring Security with diagrams.
2. How does Spring Security authenticate a user from a database?
3. Explain URL-based and method-level authorization with examples.

If you want, I can also give:

- ✓ ER Diagram for User–Role Database
- ✓ Login + Register API with DB
- ✓ Spring Security + JWT Authentication
- ✓ Full mini-project (REST API with role-based security)

📄 👍 💬 ↗️ ↺ ...

□ JWT Authorization

Below is a **clear, exam-oriented, easy-to-remember** explanation of **JWT Authorization in Spring Boot** with flow diagrams, definitions, steps, and important exam questions.

This is one of the *most important Spring Security topics* for university exams and interviews.

JWT Authorization in Spring Boot (Exam-Oriented Explanation)

JWT stands for **JSON Web Token**.

It is used to secure REST APIs using **token-based authentication and authorization**.

Unlike session-based login, JWT is **stateless**, meaning the server does *not store session data*.

All required information is stored in the **token itself**.

=====

1. What is JWT?

=====

JWT is a **digitally signed token** (string) that contains user information like:

- username
- roles
- expiration time
- issuer

JWT Format:

css

 Copy code

```
Header.Payload.Signature
```

Example:

 Copy code

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbWV9LmF1dG8yZ3xyZ
```

=====

2. Why JWT? (Exam Points)

=====

- ✓ Stateless → No sessions on server
 - ✓ Lightweight → Perfect for mobile + REST
 - ✓ Scalable → Multiple servers can validate same token
 - ✓ Secure → Signed using secret key
 - ✓ Fast → No DB lookup for every request
-

=====

★ 3. JWT Authentication & Authorization Flow

=====

Step 1: Login

User sends POST request with username & password:

```
bash
```

 Copy code

```
POST /login
```

Step 2: Authentication

Spring Security validates credentials using DB → If valid, generates JWT.

Step 3: JWT Returned to Client

Token is sent in response:

```
makefile
```

 Copy code

```
Authorization: Bearer <jwt-token>
```

Step 4: Authorization

For each API request, client sends the token in header:

```
makefile
```

 Copy code

```
Authorization: Bearer eyJhbGciOiJIUz...
```

Step 5: Token Validation

JWT Filter validates token signature + expiration.

If valid → Request forwarded to Controller.

If invalid → 401 Unauthorized.

4. JWT Authorization Filter (Key Component)

A custom filter checks JWT token in each request:

java

 Copy code

```
public class JwtAuthFilter extends OncePerRequestFilter {

    @Autowired
    private JwtService jwtService;

    @Autowired
    private UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {

            String token = authHeader.substring(7);
            String username = jwtService.extractUsername(token);

            UserDetails userDetails = userDetailsService.loadUserByUsername(username);

            if (jwtService.isTokenValid(token, userDetails)) {
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
            filterChain.doFilter(request, response);
        }
    }
}
```

=====

5. Security Configuration for JWT Authorization

=====

java

 Copy code

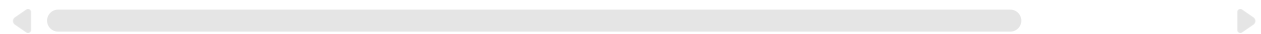
```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/login", "/auth/register").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPol:
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter

        return http.build();
    }
}
```



=====

6. Core Components in JWT Authorization

=====

✓ (1) JwtService

- Generates token
- Validates token

- Extracts username

✓ (2) JwtAuthFilter

- Reads token from header
- Validates token
- Sets authentication in Spring context

✓ (3) UserDetailsService

- Loads user details from DB

✓ (4) SecurityFilterChain